ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA <TÊN KHOA>



BÁO CÁO ĐỀ TÀI
MÔN KHOA HỌC DỮ LIỆU

DỰ ĐOÁN GIÁ Ô TÔ TRÊN CHỢ TỐT
BẰNG MÔ HÌNH RANDOM FOREST

Giảng viên hướng dẫn:	<Tên giảng viên>
Sinh viên thực hiện:	<Họ tên SV 1> – <MSSV>
	<Họ tên SV 2> – <MSSV>
Lớp học phần:	<Mã lớp>

Đà Nẵng, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 8
1.1. Bối cảnh 8
1.2. Phát biểu bài toán 8
1.3. Mục tiêu và phạm vi 8
1.4. Quy trình thực hiện 9
1.5. Công cụ sử dụng 9
CHƯƠNG 2. THU THẬP DỮ LIỆU 10
2.1. Nguồn dữ liệu và phương pháp thu thập 10
2.2. Thu thập có trách nhiệm 10
2.3. Từ điển dữ liệu 10
2.4. Cơ chế thu thập và hệ quả cho tiền xử lý 11
CHƯƠNG 3. KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU 12
3.1. Khám phá dữ liệu ban đầu 12
3.2. Chuẩn hóa kiểu dữ liệu và giá trị giả 14
3.3. Trích vị trí, loại cột thừa và tin trùng 15
3.4. Giá trị bất hợp lý 15
3.5. Giá trả trước / đặt cọc 16
3.6. Xử lý giá trị thiếu 16
3.7. Biến đổi log cho giá 17
3.8. Gộp nhãn hiếm theo cấp 18
3.9. Tạo đặc trưng 19
3.10. Dữ liệu mất cân bằng 19
3.11. Tổng kết tiền xử lý 20
CHƯƠNG 4. XÂY DỰNG MÔ HÌNH RANDOM FOREST 22
4.1. Thiết kế thí nghiệm 22
4.1.1. Chia dữ liệu 22
4.1.2. Biến mục tiêu và đặc trưng 23
4.1.3. Thước đo đánh giá 23

4.2. Chuẩn bị đặc trưng: one-hot hay mã số thứ tự? 24
4.3. Cơ sở lý thuyết của Random Forest 25
4.3.1. Thành phần: cây hồi quy CART....25
4.3.2. Rừng ngẫu nhiên....26
4.4. Random Forest: bản thư viện 27
4.5. Random Forest: bản tự cài đặt bằng NumPy....28
4.5.1. Cấu trúc 28
4.5.2. Kiểm chứng thành phần: một cây....28
4.5.3. Tinh chỉnh bằng OOB 28
4.5.4. Từ một cây đến một rừng 29
4.6. Mô hình so sánh: Ridge Regression 30
4.7. Kỹ thuật xử lý mất cân bằng....30
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ....32
5.1. Kết quả tổng thể trên tập test....32
5.2. Sai số theo phân khúc giá 33
5.3. Dự đoán so với giá thật và phần dư....34
5.4. Những xe bị đoán sai nhiều nhất 34
5.5. Kiểm định ý nghĩa thống kê bằng bootstrap....35
5.6. Rừng tự cài đặt so với rừng thư viện trên từng xe....36
5.7. Diễn giải mô hình 37
5.7.1. Độ quan trọng theo mức giảm sai số (impurity) 37
5.7.2. Kiểm tra lại bằng permutation importance....37
5.7.3. Đường khấu hao 39
5.8. Thí nghiệm log giá và xử lý mất cân bằng 39
5.9. Ảnh hưởng của tin đăng lại 40
5.10. Lưu và sử dụng mô hình....41
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 42
6.1. Kết quả đạt được....42
6.2. Hạn chế....42
6.3. Hướng phát triển....43

TÀI LIỆU THAM KHẢO 44
PHỤ LỤC 45
A. Cấu trúc mã nguồn.....45
B. Cách chạy.....45
C. Phân công công việc 45

DANH MỤC HÌNH

Hình 3.1. Mười hãng xe có nhiều tin đăng nhất (dữ liệu thô) 12
Hình 3.2. Phân bố giá rao bán trên thang gốc và thang log 12
Hình 3.3. Giá rao theo năm sản xuất 13
Hình 3.4. Phân bố giá theo hãng xe 13
Hình 3.5. Quan hệ giữa số km đã đi và giá (mật độ tin đăng) 14
Hình 3.6. Tỉ lệ giá trị thiếu của từng cột và cách xử lý 17
Hình 3.7. Phân bố giá gốc và log giá sau tiền xử lý 18
Hình 3.8. Số tin đăng theo phân khúc giá 20
Hình 3.9. Số tin còn lại qua từng bước làm sạch 21
Hình 4.1. Chia train/test phân tầng: tỉ lệ các phân khúc giá như nhau ở hai tập 22
Hình 4.2. MAPE tổng và MAPE xe trên 2 tỷ theo cách mã hóa 25
Hình 4.3. Một cây hồi quy độ sâu 3 học từ tập train (ô cam: lá, kèm giá dự đoán) 26
Hình 4.4. OOB RMSE của rừng tự cài đặt theo số cây 29
Hình 4.5. Một cây so với cả rừng trên tập test (hai bản cài đặt) 30
Hình 5.1. So sánh MAPE và tỉ lệ xe được định giá sát giữa các mô hình 32
Hình 5.2. MAPE theo phân khúc giá 33
Hình 5.3. Giá dự đoán so với giá thật trên tập test (thang log) 34
Hình 5.4. Phân bố phần dư và sai số tuyệt đối theo phân khúc 34
Hình 5.5. MAPE trên tập test với khoảng tin cậy 95% (bootstrap) 36
Hình 5.6. Dự đoán của rừng tự cài đặt so với rừng thư viện trên từng xe trong tập test 36
Hình 5.7. Độ quan trọng đặc trưng (impurity): bản thư viện và bản tự cài đặt 37
Hình 5.8. Hai cách đo độ quan trọng đặc trưng của Random Forest 38
Hình 5.9. Giá dự đoán theo tuổi xe của ba mẫu xe ở ba phân khúc 39
Hình 5.10. MAPE tổng và MAPE theo phân khúc của các phương án (CV trên train) 39

DANH MỤC BẢNG

Bảng 1.1. Các bước thực hiện đề tài 9
Bảng 2.1. Từ điển dữ liệu thô 10
Bảng 2.2. Cơ chế thu thập dữ liệu và cách xử lý tương ứng.....11
Bảng 3.1. Các giá trị giả trong dữ liệu thô.....14
Bảng 3.2. Quy tắc phát hiện giá trị bất hợp lý 15
Bảng 3.3. Ví dụ tin bị loại vì nghi ghi giá trả trước / đặt cọc 16
Bảng 3.4. Cơ chế thiếu và cách điền cho từng cột.....17
Bảng 3.5. Quy tắc gộp nhãn hiếm theo cấp 18
Bảng 3.6. So sánh các cách gộp nhãn (5-fold CV trên train, MAPE) 19
Bảng 3.7. Phân bố dữ liệu theo phân khúc giá.....20
Bảng 3.8. Số dòng qua từng bước tiền xử lý.....20
Bảng 4.1. Đặc trưng đầu vào của hai mô hình.....23
Bảng 4.2. Các thước đo đánh giá và cách đọc 24
Bảng 4.3. Hai cách mã hóa biến phân loại cho cây 24
Bảng 4.4. So sánh cách mã hóa (5-fold CV trên train, MAPE).....25
Bảng 4.5. Siêu tham số của Random Forest 27
Bảng 4.6. Năm cấu hình Random Forest tốt nhất (thư viện, 3-fold CV).....27
Bảng 4.7. Các lớp của Random Forest tự cài đặt.....28
Bảng 4.8. Kiểm chứng cây hồi quy tự cài đặt.....28
Bảng 4.9. Tinh chỉnh Random Forest tự cài đặt bằng OOB (30 cây mỗi cấu hình).....29
Bảng 4.10. Một cây so với cả rừng trên tập test 30
Bảng 4.11. Các kỹ thuật xử lý mất cân bằng được thử nghiệm.....31
Bảng 5.1. Kết quả trên tập test.....32
Bảng 5.2. MAPE theo phân khúc giá trên tập test.....33
Bảng 5.3. Mười xe bị đoán sai nhiều nhất trên tập test (giá: triệu đồng).....34
Bảng 5.4. Hiệu MAPE giữa các mô hình (điểm %), bootstrap ghép cặp 1.000 lần 35
Bảng 5.5. Tóm tắt so sánh hai bản Random Forest 36
Bảng 5.6. Độ quan trọng đặc trưng của Random Forest (thư viện) theo hai cách đo...38

Bảng 5.7. Thí nghiệm trên Random Forest (5-fold CV trên tập train) 39
Bảng 5.8. Sai số trên nhóm tin có và không có bản sao trong tập train.....40
Bảng 5.9. MAPE (CV trên train) khi chia ngẫu nhiên và chia theo nhóm 41
Bảng 5.10. Các file mô hình đã lưu trong thư mục model/.....41
Bảng 5.11. Ví dụ dự đoán giá (triệu đồng) 41
Bảng PL.1. Cấu trúc thư mục dự án 45
Bảng PL.2. Phân công công việc trong nhóm 45

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Tiếng Anh	Ý nghĩa
MAPE	Mean Absolute Percentage Error	Sai số phần trăm tuyệt đối trung bình (thước đo chính)
MedAPE	Median Absolute Percentage Error	Trung vị sai số phần trăm tuyệt đối
MAE	Mean Absolute Error	Sai số tuyệt đối trung bình (triệu đồng)
RMSE	Root Mean Squared Error	Căn bậc hai sai số bình phương trung bình
R^2	Coefficient of determination	Hệ số xác định
CV	Cross-validation	Kiểm định chéo (K-fold)
OOB	Out-of-bag	Các mẫu không được chọn vào mẫu bootstrap của một cây
CART	Classification and Regression Tree	Cây phân loại và hồi quy
RF	Random Forest	Rừng ngẫu nhiên
SSE	Sum of Squared Errors	Tổng bình phương sai số
KTC	Confidence interval	Khoảng tin cậy
EDA	Exploratory Data Analysis	Phân tích khám phá dữ liệu

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh

Thị trường ô tô cũ ở Việt Nam ngày càng sôi động, và phần lớn giao dịch bắt đầu từ các trang rao vặt trực tuyến như Chợ Tốt (`xe.chotot.com`). Người bán thường tự đặt giá theo cảm tính hoặc tham khảo vài tin tương tự, còn người mua khó biết một mức giá rao là hợp lý hay không. Một mô hình ước lượng giá từ các thông tin cơ bản của xe (hãng, dòng xe, năm sản xuất, số km, hộp số, nhiên liệu, kiểu dáng...) giúp cả hai phía có một mốc tham chiếu khách quan.

Đề tài được thực hiện theo yêu cầu của môn Khoa học dữ liệu: xây dựng **một mô hình học máy từ đầu đến cuối**, từ thu thập dữ liệu, tiền xử lý, huấn luyện đến đánh giá bằng các thước đo phù hợp, có áp dụng **kỹ thuật xử lý dữ liệu mất cân bằng**. Mô hình phải được **phân tích kỹ** và **tự cài đặt** thuật toán, không chỉ gọi thư viện. Đề tài chọn **Random Forest** làm mô hình chính.

1.2. Phát biểu bài toán

Đây là bài toán **hồi quy có giám sát**:

- **Đầu vào:** các thuộc tính của một tin rao bán ô tô: hãng, dòng xe, xuất xứ, tình trạng (mới / đã sử dụng), hộp số, nhiên liệu, kiểu dáng, số chỗ, năm sản xuất, số km đã đi.
- **Đầu ra:** giá rao bán dự đoán (triệu đồng).
- **Dữ liệu:** 14.835 tin rao bán ô tô thu thập từ Chợ Tốt ngày 25/10/2026; sau tiền xử lý còn 13.937 tin.
- **Thước đo chính:** MAPE (sai số phần trăm trung bình), báo cáo kèm theo từng phân khúc giá.

1.3. Mục tiêu và phạm vi

1. Thu thập dữ liệu tin rao bán ô tô từ Chợ Tốt bằng web scraping.
2. Làm sạch dữ liệu: xử lý giá trị thiếu, giá trị bất hợp lý, giá giả, giá trả trước, tin trùng; gộp nhãn hiếm; tạo đặc trưng.
3. Xây dựng mô hình chính **Random Forest** với hai bản: dùng thư viện `scikit-learn` và **tự cài đặt bằng NumPy**, rồi kiểm chứng hai bản cho kết quả tương đương. **Ridge Regression** (thư viện) chỉ dùng làm mô hình so sánh.
4. Đánh giá bằng nhiều thước đo, kiểm định ý nghĩa thống kê của chênh lệch giữa các mô hình.

5. Thử nghiệm các kỹ thuật xử lý mất cân bằng (trọng số mẫu, oversampling, undersampling) và đánh giá tác động.
6. Diễn giải mô hình: yếu tố nào quyết định giá xe, xe mất giá theo tuổi như thế nào.

Phạm vi: giá trong dữ liệu là **giá rao bán**, không phải giá giao dịch thực tế. Dữ liệu là ảnh chụp thị trường tại một thời điểm, không xét biến động giá theo thời gian.

1.4. Quy trình thực hiện

Đề tài đi theo quy trình chuẩn của một dự án khoa học dữ liệu, mỗi bước tương ứng với một notebook:

Bảng 1.1. *Các bước thực hiện đề tài*

Bước	Nội dung	File	Kết quả
1. Thu thập	Crawl 750 trang danh sách tin, đọc thuộc tính từng tin	`crawl_data.ipynb`	`raw_data_oto.csv` (14.835 dòng, 14 cột)
2. Khám phá	Biểu đồ phân bố, tương quan, giá trị thiếu	`visualize_raw_data.ipynb`	Biểu đồ trong `figures/`
3. Tiền xử lý	Làm sạch, điền thiếu, gộp nhãn hiếm, tạo đặc trưng	`preprocess_data.ipynb`	`clean_data_oto.csv` (13.937 dòng, 18 cột)
4. Mô hình	Random Forest (thư viện + tự cài đặt), Ridge để so sánh, đánh giá, thí nghiệm	`model.ipynb`	Mô hình đã lưu trong `model/`
5. Triển khai	Nạp mô hình, dự đoán từ thông tin thô của xe	`model/price_model.py`	Hàm `predict_price()`

1.5. Công cụ sử dụng

- **Ngôn ngữ:** Python 3, môi trường Jupyter Notebook.
- **Thu thập dữ liệu:** `requests`, `BeautifulSoup`.
- **Xử lý và trực quan hóa:** `pandas` 3.0, `NumPy`, `matplotlib`.
- **Học máy:** `scikit-learn` 1.9 (bản thư viện, dùng làm chuẩn đối chiếu); `NumPy` (bản tự cài đặt); `joblib` để lưu mô hình.

CHƯƠNG 2. THU THẬP DỮ LIỆU

2.1. Nguồn dữ liệu và phương pháp thu thập

Dữ liệu được thu thập từ chuyên mục mua bán ô tô của Chợ Tốt (`https://xe.chotot.com/mua-ban-oto`) bằng notebook `crawl_data.ipynb`, hoàn tất ngày **25/10/2026**. Quy trình gồm hai tầng:

1. **Duyệt trang danh sách:** lần lượt truy cập 750 trang danh sách tin mới nhất (`?page=1` đến `?page=750`), lấy đường dẫn của từng tin.
2. **Đọc trang chi tiết:** với mỗi tin, tải trang và đọc các thẻ HTML có thuộc tính `itemprop` tương ứng với từng thuộc tính của xe (riêng giá đọc từ thẻ `name="price"`). Kết quả được ghi nối tiếp vào `raw_data_oto.csv`.

Code có cơ chế **chạy tiếp**: các URL đã có trong file được bỏ qua, nên nếu bị ngắt giữa chừng chỉ cần chạy lại. Các tin "ưu tiên" lặp lại ở đầu mọi trang cũng chỉ được ghi một lần. Kết quả là **14.835 tin** với 14 cột (số thứ tự, URL và 12 thuộc tính).

2.2. Thu thập có trách nhiệm

- Chỉ lấy thông tin công khai trên trang tin rao, không đăng nhập, không lấy thông tin liên hệ của người bán; dữ liệu chỉ dùng cho mục đích học tập.
- Nghỉ giữa các yêu cầu (0,5 giây giữa các tin, 1 giây giữa các trang) và tự lùi thời gian khi máy chủ trả về mã 403/429, để không gây tải cho máy chủ.
- Hạn chế kỹ thuật: danh sách tin được tìm bằng một class CSS sinh tự động; khi trang đổi giao diện cần sửa lại bộ chọn.

2.3. Từ điển dữ liệu

Bảng 2.1 mô tả các cột thu được. Cột *"Người bán nhập thế nào"* rất quan trọng cho bước tiền xử lý: cột **chọn từ danh sách** có tập giá trị cố định, viết thống nhất; còn cột **gõ tự do** (số km, giá) là nguồn lỗi nhập liệu chính.

Bảng 2.1. *Từ điển dữ liệu thô*

Cột	Ý nghĩa	Người bán nhập	Sau xử lý
`mileage_v2`	Số km đã đi	**gõ tự do** (dạng 109.366)	số (`mileage_km`)
`carorigin`	Xuất xứ	chọn từ danh sách, không bắt buộc	phân loại
`condition_ad`	Tình trạng (Mới / Đã sử dụng)	chọn từ danh sách	phân loại
`veh_warranty_policy`	Chính sách bảo hành	(một giá trị duy nhất)	bỏ
`carbrand`	Hãng xe	chọn từ danh sách	phân loại

Cột	Ý nghĩa	Người bán nhập	Sau xử lý
`carmodel`	Dòng xe	chọn theo hãng	phân loại
`mfdate`	Năm sản xuất	chọn (có mục "trước năm 1980")	số
`gearbox`	Hộp số	chọn (3 giá trị)	phân loại
`fuel`	Nhiên liệu	chọn (4 giá trị)	phân loại
`cartype`	Kiểu dáng	chọn (9 giá trị), không bắt buộc	phân loại
`carseats`	Số chỗ ngồi	chọn (có mục "Khác"), không bắt buộc	số
`price`	Giá rao bán (đồng)	**gõ tự do**	biến mục tiêu
`url`	Đường dẫn tin	sinh tự động, chứa quận/huyện và tỉnh	trích `province`, rồi bỏ

2.4. Cơ chế thu thập và hệ quả cho tiền xử lý

Hiểu **cách dữ liệu được tạo ra** giúp biết mỗi loại lỗi đến từ đâu và nên xử lý thế nào:

Bảng 2.2. *Cơ chế thu thập dữ liệu và cách xử lý tương ứng*

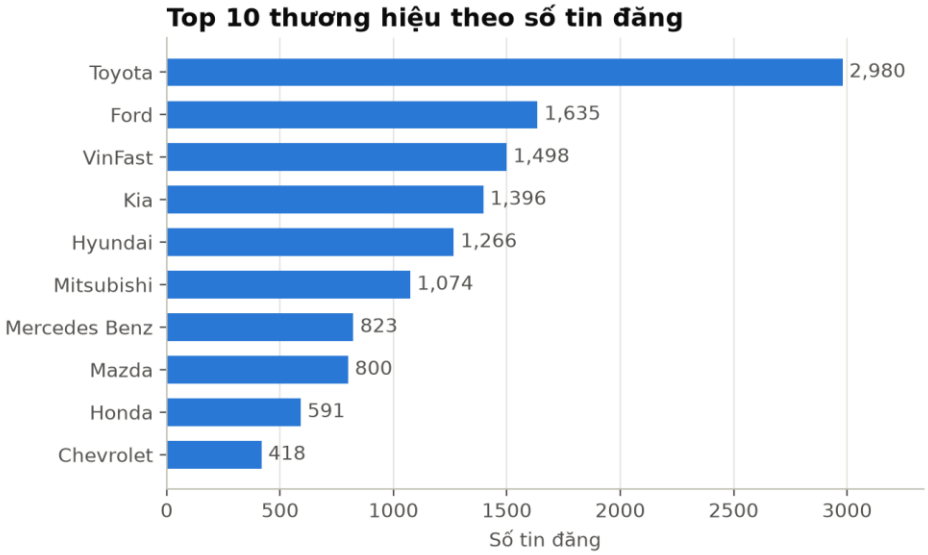
Cơ chế	Hệ quả	Cách xử lý
Không tìm thấy thẻ của một thuộc tính → ghi chuỗi `"N/A"`	`N/A` là thiếu thật; pandas mặc định tự đổi thành NaN, làm mất dấu vết	Đọc với `keep_default_na=False`, rồi tự đổi về NaN
Trang hiển thị `"Đang cập nhật"` khi người bán không chọn xuất xứ	Cũng là thiếu thật	Đổi về NaN
Danh sách có sẵn mục `"Khác"`, `"Dòng khác"`, `"Nước khác"`	Là lựa chọn thật của người bán	Giữ làm nhãn (trừ `carseats = "Khác"`)
Số km và giá là ô gõ tự do	Gõ thiếu/thừa số 0, ghi tiền trả trước, giá giả 123.456.789	Quy tắc kiểm tra ở mục 3.4, 3.5
Xe "Mới" không có ô số km	Số km thiếu ở mọi xe mới	Điền 0 km
Bỏ qua URL đã gặp	Mỗi dòng là một tin khác nhau; dòng trùng nội dung là tin đăng lại	Loại trùng theo nội dung
URL dạng `mua-ban-oto-<quận>-<tỉnh>/<mã>.htm`	Có thông tin vị trí	Trích tỉnh/thành
Một lần thu thập, 750 trang mới nhất	Ảnh chụp thị trường tại một thời điểm; giá là giá rao	Tuổi xe tính theo năm 2026

CHƯƠNG 3. KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU

Chương này trình bày các bước biến dữ liệu thô thành tập dữ liệu sạch cho mô hình (notebook `preprocess_data.ipynb`). Mỗi quyết định đều kèm lý do, và đều xuất phát từ việc **giá là biến cần dự đoán**.

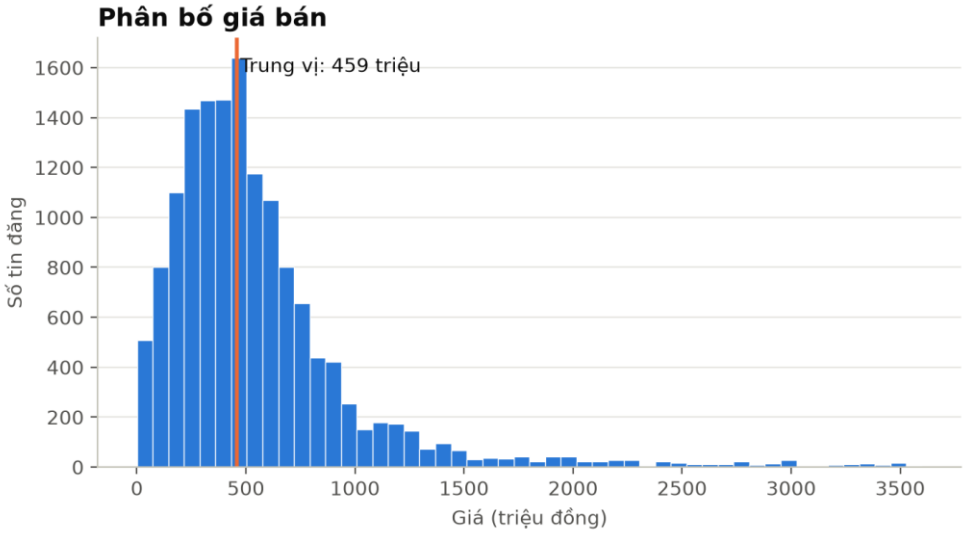
3.1. Khám phá dữ liệu ban đầu

Dữ liệu thô có 14.835 dòng và 14 cột. Thị trường tập trung vào các hãng phổ thông: Toyota chiếm khoảng 20% số tin, tiếp theo là Ford, VinFast, Kia, Hyundai (Hình 3.1).



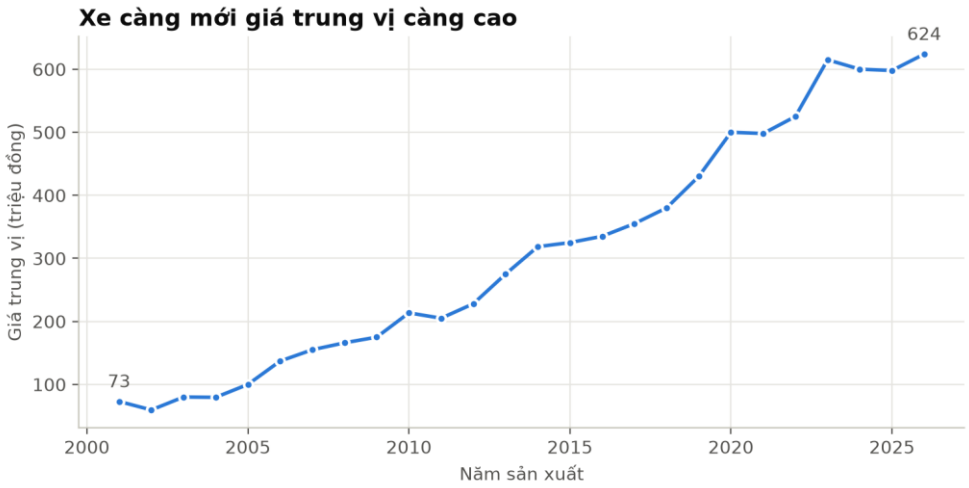
Hình 3.1. *Mười hãng xe có nhiều tin đăng nhất (dữ liệu thô)*

Giá rao **lệch phải rất mạnh** (độ lệch skewness = 5,08): đa số xe có giá 200-800 triệu, nhưng có một đuôi dài các xe vài tỷ đến hàng chục tỷ đồng (Hình 3.2). Đặc điểm này ảnh hưởng trực tiếp đến cách chọn biến mục tiêu (mục 3.7).

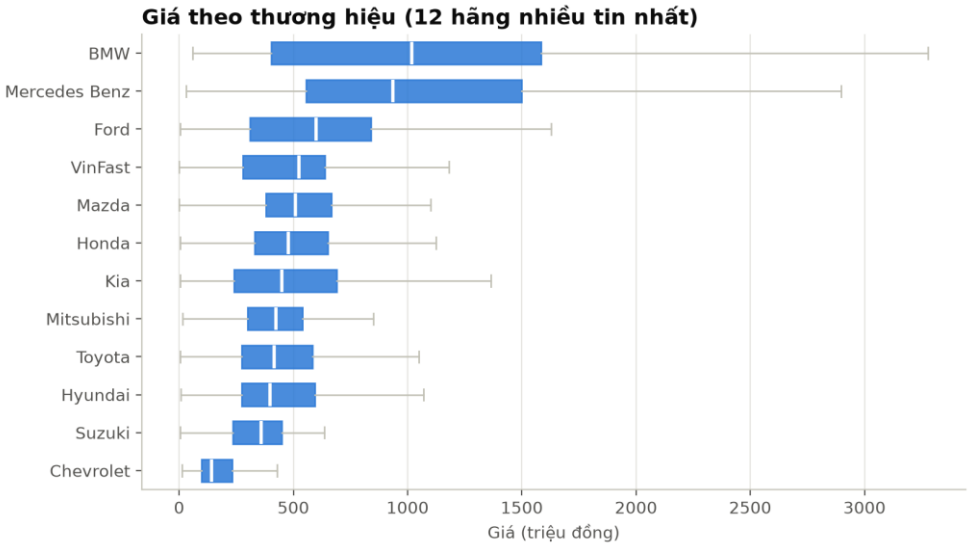


Hình 3.2. *Phân bố giá rao bán trên thang gốc và thang log*

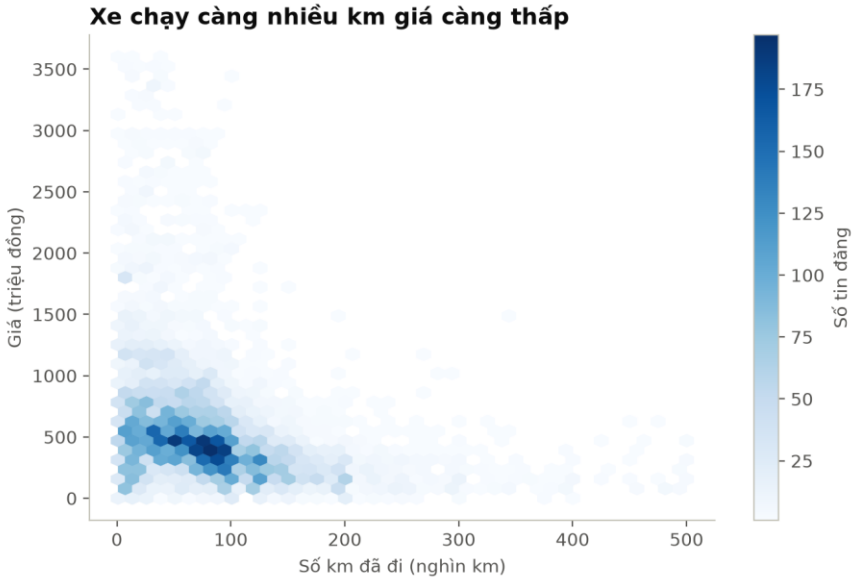
Hai yếu tố có quan hệ rõ nhất với giá là **năm sản xuất** và **số km**: xe càng mới giá càng cao, xe chạy càng nhiều km giá càng thấp, và quan hệ không tuyến tính (xe mất giá nhanh trong những năm đầu). Giá cũng chênh lệch lớn giữa các hãng.



Hình 3.3. *Giá rao theo năm sản xuất*



Hình 3.4. *Phân bố giá theo hãng xe*



Hình 3.5. *Quan hệ giữa số km đã đi và giá (mật độ tin đăng)*

3.2. Chuẩn hóa kiểu dữ liệu và giá trị giả

Bảng 3.1 thống kê các giá trị "giả" trong dữ liệu thô, tức các chuỗi không phải giá trị thật của thuộc tính:

Bảng 3.1. *Các giá trị giả trong dữ liệu thô*

Cột	`N/A` (không có thẻ)	`Đang cập nhật`	`Khác`	Số giá trị khác nhau
mileage_v2	1.811	0	0	1.995
carorigin	0	6.012	0	11
veh_warranty_policy	0	0	0	1
cartype	4.883	0	0	10
carseats	3.028	0	659	12
price	1	0	0	1.433

- `N/A` và `Đang cập nhật` đều là người đăng **không điền thông tin**, nên được đổi thành giá trị thiếu (NaN). Nếu để nguyên, mô hình sẽ coi "Đang cập nhật" là một xuất xứ thật.
- `Khác` trong `carseats` không đổi ra số được nên coi là thiếu; còn `Dòng khác`, `Kiểu dáng khác` là nhãn có nghĩa nên giữ nguyên.
- `price`, `mileage_v2`, `carseats` dạng chuỗi `1.095.000.000` được bỏ dấu chấm rồi ép sang số; giá đổi sang đơn vị triệu đồng (`price_tr`).
- `mfdate = "trước năm 1980"` (12 dòng) được gán 1979.

3.3. Trích vị trí, loại cột thừa và tin trùng

Vị trí từ URL. Mọi URL có dạng `mua-ban-oto-<quận/huyện>-<tỉnh>/<mã tin>.htm`. So khớp phần cuối với danh sách 63 tỉnh/thành để lấy cột `province` (giữ dạng slug của Chợ Tốt như `tp-ho-chi-minh`, `ha-noi`). Trích được tỉnh cho 100% số tin. Không tách quận/huyện vì có 453 quận/huyện, trung vị chỉ 4 tin mỗi quận, 315 quận dưới 10 tin: dữ liệu quá thưa. Ngay cả ở mức tỉnh, TP.HCM và Hà Nội đã chiếm 67% số tin, 28 tỉnh có dưới 30 tin.

Cột loại bỏ: `no` và `url` là định danh, không mang thông tin dự đoán; `veh_warranty_policy` chỉ có một giá trị duy nhất ("Bảo hành hãng") ở mọi dòng.

Tin trùng: 691 dòng (4,7%) giống hệt nhau ở mọi thuộc tính và giá (331 trong số đó là xe mới, thường do đại lý đăng nhiều xe cùng cấu hình). Giữ lại sẽ làm lệch phân phối, và khi chia train/test thì cùng một xe có thể nằm ở cả hai phía, làm điểm đánh giá cao ảo. Chỉ giữ một bản, còn lại **14.144 dòng**.

3.4. Giá trị bất hợp lý

Bước này làm **trước** khi điền thiếu, để không điền dựa trên số liệu sai. Phân biệt hai loại: **sai dữ liệu** (nhập nhầm, không thể xảy ra) thì loại hoặc sửa; **ngoại lệ thật** (xe siêu sang giá chục tỷ) thì giữ, vì đó là xe có thật.

Bảng 3.2. *Quy tắc phát hiện giá trị bất hợp lý*

Cột	Quy tắc	Lý do	Số dòng
Giá	Loại nếu < 20 triệu hoặc > 10 tỷ	Gõ thiếu/thừa số 0; không biết giá đúng nên loại	64
Giá	Loại giá là dãy số liên tiếp (123.456.789...)	Giá giả khi người bán không muốn công khai giá	14
Tình trạng	"Mới" nhưng hơn 3 năm tuổi → "Đã sử dụng"	Ghi sai tình trạng (ví dụ Tucson 2011 "mới")	6 (sửa)
Số km	Xe đã dùng > 500.000 km → thiếu	Nhập thừa số 0	106
Số km	Xe ≥ 2 năm tuổi mà < 100 km → thiếu	Nhập theo đơn vị nghìn km	447
Số km	> 100.000 km/năm → thiếu	Bất khả thi với xe dân dụng	25

Các giá như 399.999.999 hay 888.888.888 vẫn được **giữ**, vì đó là cách định giá tâm lý hoặc chọn số đẹp, vẫn là giá thật. Sau bước này còn **14.066 dòng** (loại 78 dòng); 578 giá trị số km bất hợp lý được chuyển thành thiếu để điền lại ở mục 3.6.

3.5. Giá trả trước / đặt cọc

Ngưỡng 20 triệu chỉ bắt được lỗi gõ thiếu số. Vẫn còn những tin mà "giá" thực ra là **số tiền trả trước hoặc đặt cọc**, ví dụ xe đời 2025-2026 còn mới mà giá chỉ 30-100 triệu. Một ngưỡng tuyệt đối không bắt được trường hợp này, vì 100 triệu hợp lý với xe 20 năm tuổi nhưng vô lý với xe 1 năm tuổi.

Quy tắc: loại tin có giá **thấp hơn 30% giá trung vị của các xe cùng hãng, dòng và năm sản xuất** (chỉ xét nhóm có ít nhất 3 xe). Không xe nào mất 70% giá so với xe cùng đời chỉ vì tình trạng hay số km. Kết quả: loại **129 tin**, 94% trong số đó có giá dưới 200 triệu; còn lại **13.937 dòng**.

Bảng 3.3. *Ví dụ tin bị loại vì nghi ghi giá trả trước / đặt cọc*

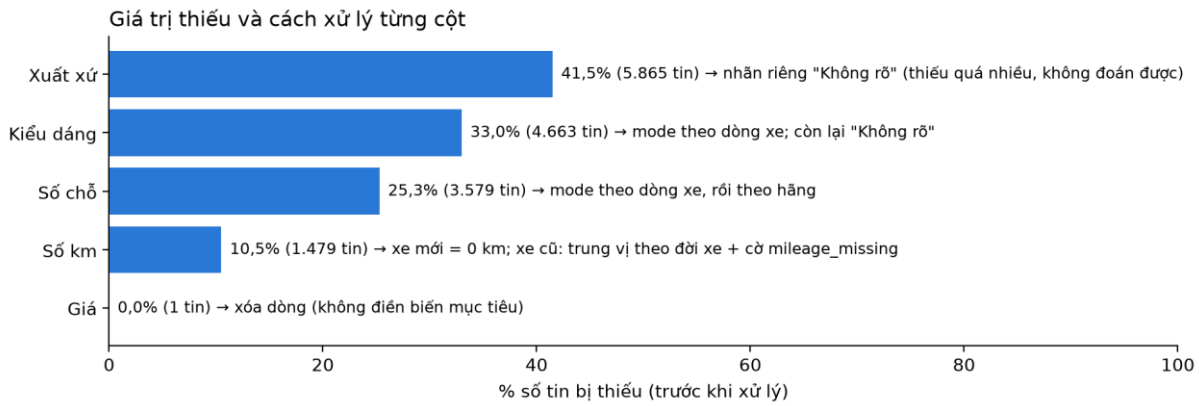
Xe	Năm	Tình trạng	Giá rao (triệu)	Trung vị nhóm (triệu)
Toyota Camry	2026	Đã sử dụng	300	1.400
Ford Everest	2026	Đã sử dụng	320	1.325
Toyota Fortuner	2026	Mới	125	1.249
Hyundai Santa Fe	2025	Mới	220	959
Mercedes Benz E Class	2019	Đã sử dụng	156	889

Đây là bước **sửa lỗi nhãn**: quy tắc dùng giá trên toàn bộ dữ liệu nhưng chỉ để loại dòng có đáp án sai, không đưa thông tin giá vào đặc trưng nên không gây rò rỉ dữ liệu vào đầu vào của mô hình.

Ngoại lệ theo IQR của log giá còn 696 dòng (5,0%), chủ yếu là xe sang thật (Mercedes, Lexus, Bentley, Range Rover...) hoặc xe cổ rất rẻ. Đây là thông tin thật của thị trường nên **giữ nguyên**; ảnh hưởng của chúng được giảm bằng biến đổi log.

3.6. Xử lý giá trị thiếu

Trước khi điền cần hiểu **vì sao thiếu**, vì cách điền phụ thuộc cơ chế thiếu.



Hình 3.6. *Tỉ lệ giá trị thiếu của từng cột và cách xử lý*

Bảng 3.4. *Cơ chế thiếu và cách điền cho từng cột*

Cột	Cơ chế thiếu	Cách xử lý
`mileage_km`	**Toàn bộ** 1.235 xe mới thiếu km (trang tin xe mới không có ô số km); chỉ 244 xe đã dùng thiếu thật, cộng 578 giá trị bất hợp lý	Xe mới: **0 km**. Xe đã dùng: trung vị theo năm sản xuất (814 xe, 5,8%), đánh dấu `mileage_missing = 1`
`carorigin`	Thiếu ~41%; một mẫu xe có thể vừa nhập khẩu vừa lắp ráp	Nhãn riêng **"Không rõ"** (5.787 dòng)
`cartype`	Thiếu ~33%; kiểu dáng gần như do dòng xe quyết định	Mode theo hãng + dòng xe (4.336); còn lại "Không rõ" (243)
`carseats`	Thiếu ~25%; số chỗ do dòng xe quyết định	Mode theo dòng xe (3.352) → theo hãng (183) → mode chung (4)
`price`	1 dòng	**Xóa** (không bịa biến mục tiêu)

Vì sao không điền trung vị số km cho xe mới? Điền trung vị sẽ biến xe mới thành "xe đã chạy 66.000 km", sai lệch nghiêm trọng. Đây là thiếu **có cấu trúc** (không đo vì không có ý nghĩa), giá trị đúng là 0.

Cột đánh dấu `mileage_missing`: giá trị điền cho xe đã dùng chỉ là "đoán trung bình". Cột này cho mô hình biết dòng nào là ước lượng để tự học mức độ tin cậy; việc người bán không khai số km cũng có thể là một tín hiệu về giá.

Vì sao `cartype` không lùi về cấp hãng còn `carseats` thì có? Trong một dòng xe, kiểu dáng gần như cố định; nhưng một hãng bán đủ loại (Ford có Ranger bán tải, Everest SUV, Focus sedan), nên gán kiểu dáng phổ biến của hãng là đoán mò. Số chỗ thì đa số là 5, đoán sai theo hãng thường chỉ lệch 5 ↔ 7 chỗ.

3.7. Biến đổi log cho giá

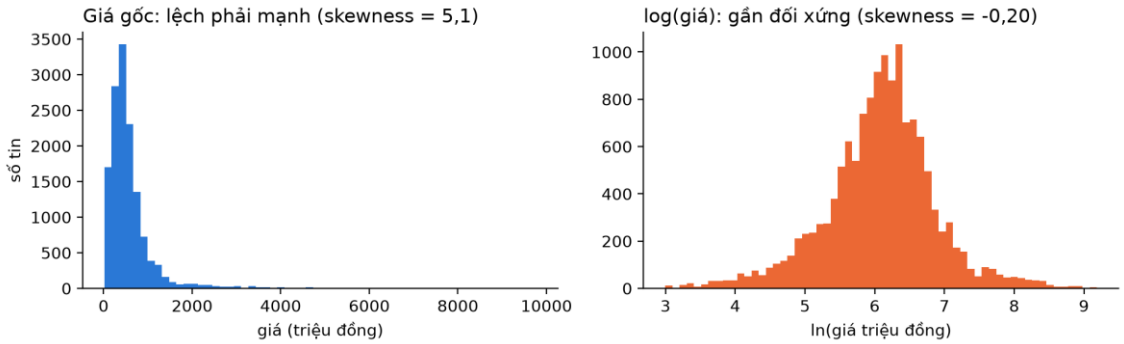
Giá lệch phải mạnh khiến mô hình hồi quy bị kéo bởi vài xe rất đắt. Lấy logarit đưa phân phối về gần đối xứng (độ lệch giảm từ **5,08 xuống −0,20**) và biến chênh lệch

tuyệt đối thành chênh lệch **tương đối**: xe 100 → 150 triệu và xe 1 → 1,5 tỷ đều là tăng 50%, khớp với cách người mua nghĩ về giá và với thước đo MAPE.

$$y = \ln(p)$$

$$\hat{p} = e^{\hat{y}}$$

Với p là giá (triệu đồng), mô hình học `price_log`, dự đoán xong dùng `exp()` để đổi về triệu đồng; **mọi thước đo đều tính trên giá thật**. Lựa chọn này được kiểm chứng bằng thí nghiệm ở mục 5.8.



Hình 3.7. *Phân bố giá gốc và log giá sau tiền xử lý*

3.8. Gộp nhãn hiếm theo cấp

Dữ liệu có 69 hãng và 509 tổ hợp hãng + dòng xe, nhiều nhãn chỉ có 1-5 tin. One-hot nguyên vẹn sinh hàng trăm cột thưa, mô hình học quy luật từ vài mẫu là học nhiễu. Cách gộp đơn giản (mọi dòng hiếm thành một nhãn `"Dòng khác"`) có vấn đề: nhãn này chiếm 17% dữ liệu nhưng tới **65% số xe trên 2 tỷ**, một chiếc Mercedes G63 và một chiếc Kia đời cũ mang cùng nhãn.

Đề tài dùng cách **gộp theo cấp**: mỗi xe lùi về nhóm gần nhất còn đủ mẫu.

Bảng 3.5. *Quy tắc gộp nhãn hiếm theo cấp*

Cấp	Quy tắc	Ví dụ
Dòng xe	Giữ `Hãng + Dòng` nếu có ≥ 10 tin	Toyota Vios
→ Hãng	Dòng hiếm → `"<Hãng> - dòng khác"`	Mercedes Benz - dòng khác
Hãng	Giữ hãng nếu có ≥ 30 tin	Volvo, Subaru
→ Nhóm hãng	Hãng hiếm → `"Hãng cao cấp khác"` / `"Hãng phổ thông khác"`	Bentley → cao cấp; Wuling → phổ thông

Nhóm cao cấp / phổ thông được phân theo **định vị thương hiệu** (hiểu biết thị trường), **không** dựa vào giá trong dữ liệu, để không dùng biến mục tiêu khi tạo đặc trưng. Kết quả: 69 hãng → **28 nhóm**, 509 dòng → **214 nhóm**; 8,9% số xe mang nhãn

"... - dòng khác". Nhóm "Hãng cao cấp khác" có 93 tin từ 13 hãng (giá trung vị 1 tỷ), "Hãng phổ thông khác" có 188 tin từ 30 hãng (giá trung vị 390 triệu).

Cách gộp và ngưỡng được **chọn bằng 5-fold CV chỉ trên tập train** (Bảng 3.6). Gộp theo cấp cải thiện rõ Ridge (MAPE 21,9% → 18,9%; xe > 2 tỷ 43,0% → 32,2%) vì mô hình tuyến tính chỉ có một hệ số cho mỗi nhãn. Random Forest được lợi ít hơn vì tự bù thông tin bằng các đặc trưng khác. Không phương án nào làm mô hình xấu đi.

Bảng 3.6. *So sánh các cách gộp nhãn (5-fold CV trên train, MAPE)*

Phương án	Ridge: MAPE	< 200 tr	> 2 tỷ	RF: MAPE	< 200 tr	> 2 tỷ
Một nhãn chung (50 / 30)	0,219	0,522	0,430	0,163	0,374	0,292
Gộp theo cấp (50 / 30)	0,206	0,474	0,392	0,163	0,361	0,278
Gộp theo cấp (30 / 10) – chọn	**0,189**	**0,449**	**0,322**	**0,161**	**0,359**	**0,287**

3.9. Tạo đặc trưng

- `age` = 2026 − năm sản xuất: tuổi xe có ý nghĩa trực tiếp hơn năm tuyệt đối.
- `km_per_year` = km / max(tuổi, 1): xe 5 năm đi 150.000 km (nhiều khả năng chạy dịch vụ) khác hẳn xe 5 năm đi 50.000 km.
- `mileage_missing`: 1 nếu số km là giá trị điền.
- `province`: tỉnh/thành của tin đăng.

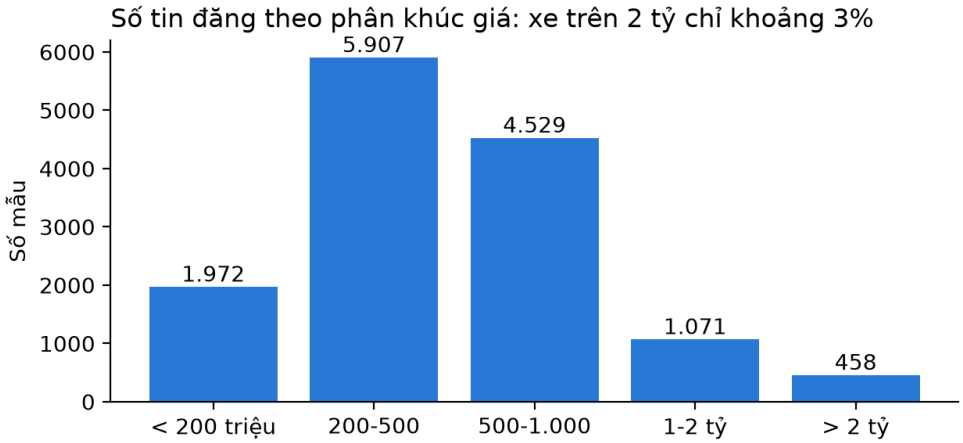
Vị trí có giúp dự đoán không? Thử thêm miền (3 nhóm) hoặc tỉnh/thành (15 nhóm, tỉnh dưới 100 tin gộp theo miền) bằng cùng cách CV: MAPE chỉ chênh ở chữ số thứ ba (Ridge 0,1895 / 0,1894 / 0,1897; Random Forest 0,1610 / 0,1604 / 0,1611), nhỏ hơn mức dao động giữa các lần chia fold. Giá rao chủ yếu do bản thân chiếc xe quyết định; chênh lệch giá giữa các miền chủ yếu do **cơ cấu xe khác nhau**, mà mô hình đã thấy qua các đặc trưng khác. Vì vậy `province` được giữ trong file để mô tả dữ liệu nhưng **không đưa vào mô hình**.

3.10. Dữ liệu mất cân bằng

Biến mục tiêu là số liên tục nên không có "lớp thiểu số" như bài toán phân loại. Mất cân bằng ở đây thể hiện ở **mật độ dữ liệu không đều theo vùng giá**: phần lớn xe ở phân khúc phổ thông, xe sang vài tỷ rất ít. Mô hình tối ưu sai số trung bình sẽ học tốt vùng đông mẫu và kém ở vùng hiếm. Để định lượng, giá được chia thành 5 phân khúc (chỉ dùng để phân tích, chia mẫu và đánh giá, không làm đặc trưng):

Bảng 3.7. *Phân bố dữ liệu theo phân khúc giá*

Phân khúc	Số tin	Tỉ lệ	Ít hơn phân khúc lớn nhất
< 200 triệu	1.972	14,1%	3,0 lần
200-500 triệu	5.907	42,4%	1,0 lần
500-1.000 triệu	4.529	32,5%	1,3 lần
1-2 tỷ	1.071	7,7%	5,5 lần
> 2 tỷ	458	3,3%	12,9 lần



Hình 3.8. *Số tin đăng theo phân khúc giá*

Phân khúc > 2 tỷ ít hơn phân khúc lớn nhất gần 13 lần: mất cân bằng thật sự. Các đặc trưng cũng lệch: xe mới chỉ 8,5%, Hybrid 2,2%, hộp số bán tự động 0,7%. Các biện pháp áp dụng:

- **Chia train/test phân tầng theo phân khúc giá**, để tập test có đủ xe ở mọi phân khúc.
- **Học trên log giá**, giảm ảnh hưởng của đuôi phải.
- **Báo cáo MAPE theo từng phân khúc**, không chỉ một con số trung bình.
- **Thử trọng số mẫu, Random Oversampling, Random Undersampling** bằng CV trên tập train (mục 4.7 và 5.8).

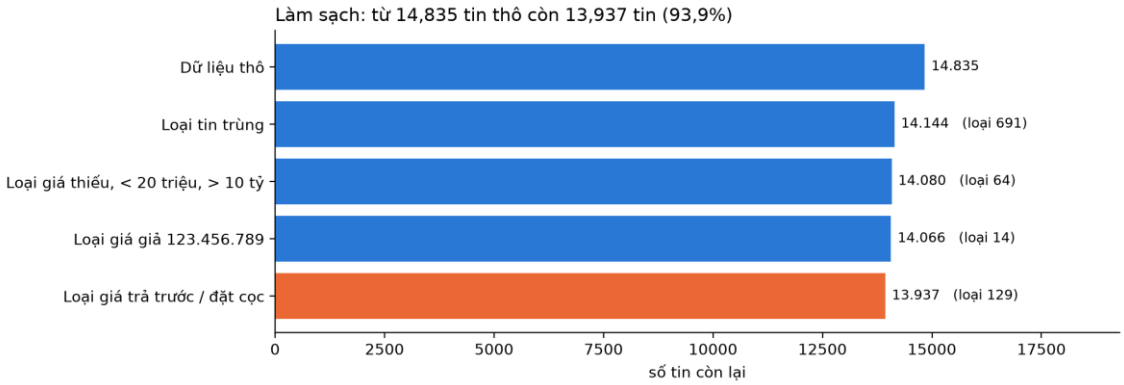
File `clean_data_oto.csv` **chỉ chứa dữ liệu thật**: nếu ghi mẫu nhân bản vào file, người dùng sau chia ngẫu nhiên sẽ khiến bản sao của cùng một xe nằm ở cả train lẫn test.

3.11. Tổng kết tiền xử lý

Bảng 3.8. *Số dòng qua từng bước tiền xử lý*

Bước	Số dòng còn lại	Thay đổi
Dữ liệu thô	14.835	

Bước	Số dòng còn lại	Thay đổi
Loại tin trùng nội dung	14.144	−691
Loại giá thiếu / < 20 triệu / > 10 tỷ / giá giả	14.066	−78
Loại giá trả trước / đặt cọc	13.937	−129
Dữ liệu sạch (giữ 93,9%)	**13.937**	**18 cột, không còn giá trị thiếu**



Hình 3.9. *Số tin còn lại qua từng bước làm sạch*

File `clean_data_oto.csv` gồm 18 cột: `carbrand`, `carbrand_grp`, `carmodel`, `carmodel_grp`, `carorigin`, `condition_ad`, `gearbox`, `fuel`, `cartype`, `carseats`, `mfdate`, `age`, `mileage_km`, `mileage_missing`, `km_per_year`, `province`, `price_tr`, `price_log`. Cột gốc `carbrand`, `carmodel` được giữ để tra cứu; mô hình dùng cột đã gộp `*_grp`.

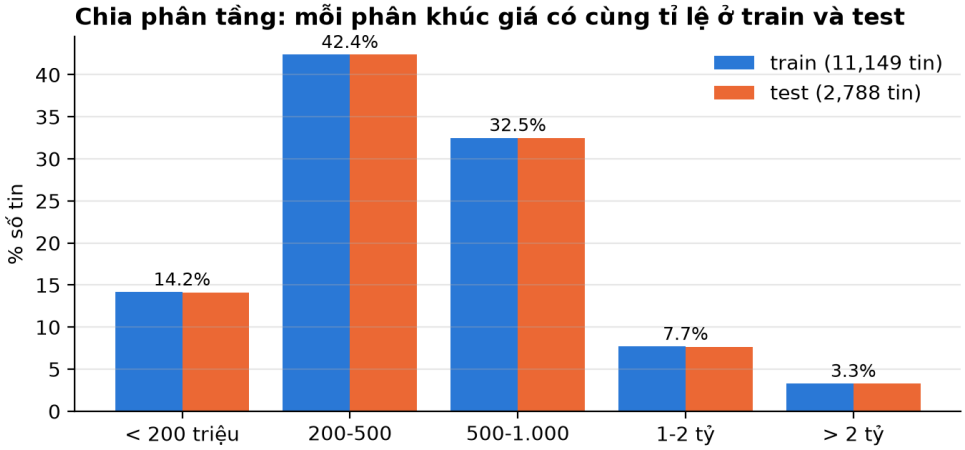
CHƯƠNG 4. XÂY DỰNG MÔ HÌNH RANDOM FOREST

Chương này trình bày mô hình chính của đề tài, **Random Forest**, với hai bản: bản dùng thư viện `scikit-learn` và bản **tự cài đặt bằng NumPy**. **Ridge Regression** chỉ đóng vai trò mô hình so sánh (mục 4.6), dùng thẳng bản thư viện. Toàn bộ nằm trong notebook `model.ipynb`.

4.1. Thiết kế thí nghiệm

4.1.1. Chia dữ liệu

Dữ liệu được chia **80% train (11.149 tin) / 20% test (2.788 tin)**, phân tầng theo phân khúc giá, `random_state = 42`. Phân tầng đảm bảo tập test có đủ xe ở mọi phân khúc: nếu chia ngẫu nhiên thông thường, tập test có thể gần như không có xe trên 2 tỷ (chỉ khoảng 3% dữ liệu), và điểm đánh giá sẽ đẹp nhưng không phản ánh khả năng với xe sang (Hình 4.1).



Hình 4.1. *Chia train/test phân tầng: tỉ lệ các phân khúc giá như nhau ở hai tập*

Không tách tập validation riêng. Tham số được chọn bằng **cross-validation** (CV) hoặc **out-of-bag** (OOB) **trên tập train**. Với chỉ khoảng 460 xe trên 2 tỷ, một tập validation cố định quá nhỏ và dễ dao động; CV dùng lần lượt mọi phần của train làm validation nên ổn định hơn. Tập test chỉ dùng **một lần** để báo cáo kết quả cuối.

Tính tái lập. Mọi bước ngẫu nhiên (chia dữ liệu, chia fold, bootstrap của rừng, chọn đặc trưng ngẫu nhiên, bootstrap khoảng tin cậy) đều cố định `random_state = 42`. Chạy lại notebook cho đúng cùng mô hình và cùng số liệu; chỉ thời gian huấn luyện thay đổi theo tải của máy.

4.1.2. Biến mục tiêu và đặc trưng

Mô hình học **ln(giá)** (mục 3.7), dự đoán xong dùng `exp()` để đổi về triệu đồng, mọi thước đo tính trên giá thật. Đặc trưng chỉ gồm thông tin người mua thấy trong tin đăng:

Bảng 4.1. *Đặc trưng đầu vào của hai mô hình*

Nhóm	Random Forest (mô hình chính)	Ridge (so sánh)
Phân loại	hãng, dòng xe (đã gộp nhãn), xuất xứ, tình trạng, hộp số, nhiên liệu, kiểu dáng; **one-hot** (mục 4.2)	như RF, one-hot
Tuổi xe	`age`	`age`, `age²`
Số km	`mileage_km`, `km_per_year` giữ nguyên	`log(1+km)`, `log(1+km/năm)`
Khác	`carseats`, `mileage_missing`	như RF
Chuẩn hóa	không cần	StandardScaler
Số cột sau mã hóa	277	278

- **Random Forest không cần biến đổi đặc trưng số:** cây chỉ so sánh ngưỡng ("tuổi ≤ 5?"), nên mọi biến đổi đơn điệu như `age²` hay `log` cho đúng cùng một phép chia. Thêm `age²` không làm mô hình tốt hơn, chỉ làm độ quan trọng của tuổi xe bị chia đôi cho hai cột. Cây cũng không phụ thuộc thang đo nên không chuẩn hóa.
- **Ridge cần `age²`, `log` và chuẩn hóa** vì chỉ cộng các hiệu ứng tuyến tính, trong khi xe mất giá nhanh những năm đầu rồi chậm lại.
- **Không dùng:** `price_tr`, `price_log` (đáp án), `mfdate` (trùng `age`), `province` (không giúp dự đoán, mục 3.9). Bộ mã hóa **fit trên train** rồi áp lên test.

4.1.3. Thước đo đánh giá

Tất cả thước đo tính trên **giá thật (triệu đồng)**:

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|p_i - \hat{p}_i|}{p_i}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|p_i - \hat{p}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(p_i - \hat{p}_i)^2}$$

trong đó p_i là giá thật và $\hat{p}_i$ là giá dự đoán của xe thứ i (triệu đồng). Bảng 4.2 tóm tắt ý nghĩa, chiều tốt và mức tham khảo của từng thước đo:

Bảng 4.2. *Các thước đo đánh giá và cách đọc*

Thước đo	Ý nghĩa	Tốt khi	Mức tham khảo
MAPE	Sai số phần trăm trung bình. **Thước đo chính**: lệch 100 triệu là nhỏ với xe 3 tỷ nhưng rất lớn với xe 150 triệu	**thấp**	< 10%: rất tốt; 10-20%: tốt; 20-50%: tạm được; > 50%: kém
MedAPE	Trung vị sai số phần trăm: sai số của một xe "điển hình", không bị vài tin sai nặng kéo lệch	**thấp**	cùng thang với MAPE
MAE	Trung bình đoán lệch bao nhiêu triệu đồng	**thấp**	không có ngưỡng chung, tùy mức giá
RMSE	Như MAE nhưng phạt nặng lỗi lớn; RMSE lớn hơn MAE nhiều nghĩa là có vài lỗi rất lớn	**thấp**	luôn ≥ MAE
R^2 (log)	Tỉ lệ biến thiên của log giá được mô hình giải thích	**cao**	1: hoàn hảo; 0: không hơn đoán trung bình; < 0: kém hơn
Lệch ≤ 10% / ≤ 20%	Tỉ lệ xe có giá dự đoán nằm trong ±10% / ±20% giá thật	**cao**	so với baseline

Thang MAPE ở cột cuối là quy ước thường dùng trong dự báo [9], không phải tiêu chuẩn cứng. Không chỉ số nào "tốt" khi đứng một mình: luôn so với **mô hình mốc (baseline)** luôn đoán giá trung vị của tập train. Ngoài ra luôn báo **MAPE theo từng phân khúc giá**, vì một con số trung bình che mất việc mô hình kém ở xe rất rẻ hoặc rất đắt.

4.2. Chuẩn bị đặc trưng: one-hot hay mã số thứ tự?

Biến phân loại phải đổi thành số trước khi đưa vào cây. Có hai cách:

Bảng 4.3. *Hai cách mã hóa biến phân loại cho cây*

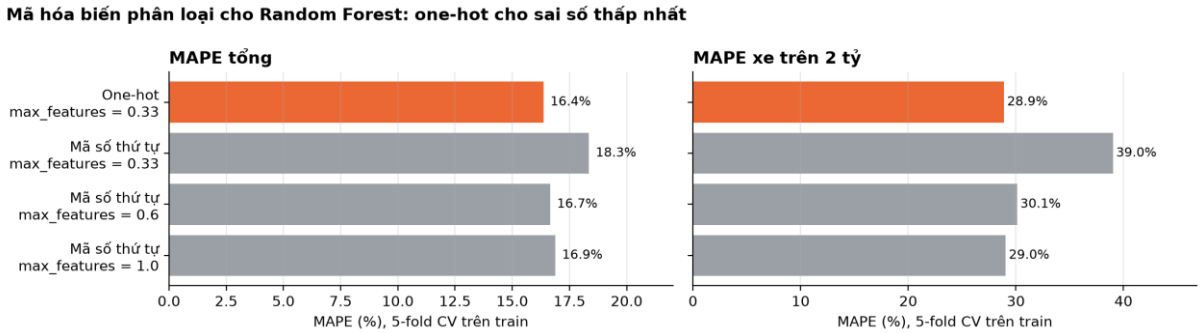
Cách	Làm gì	Ưu điểm	Nhược điểm
One-hot	Mỗi nhãn thành một cột 0/1 (dòng xe có 214 nhãn nên sinh 214 cột)	Không tạo thứ tự giả; một phép chia tách thẳng một nhãn khỏi phần còn lại	Nhiều cột (277); độ quan trọng của một đặc trưng bị chia nhỏ
Mã số thứ tự	Mỗi nhãn thành một số nguyên 0, 1, 2... trong **một** cột	Chỉ 12 cột	Thứ tự nhãn tùy ý (theo bảng chữ cái); phải chia nhiều lần mới tách được một nhãn

Với mô hình tuyến tính chỉ có one-hot là đúng, nhưng cây không bị thứ tự giả ảnh hưởng nặng, nên đây là **câu hỏi thực nghiệm**. So sánh bằng **5-fold CV phân tầng chỉ trên tập train** (bộ mã hóa fit trong phần train của từng fold), rừng 150 cây,

`max_depth = 30`; riêng `max_features` thử vài mức với mã số thứ tự vì số cột khác hẳn:

Bảng 4.4. *So sánh cách mã hóa (5-fold CV trên train, MAPE)*

Phương án	Số cột	MAPE	< 200 tr	200-500	500-1.000	1-2 tỷ	> 2 tỷ
One-hot, max_features = 0,33	**277**	**16,4%**	**36,5%**	**12,8%**	**10,8%**	**16,7%**	**28,9%**
Mã số, max_features = 0,33	12	18,4%	40,5%	14,3%	11,7%	18,8%	39,0%
Mã số, max_features = 0,6	12	16,7%	37,8%	13,1%	10,5%	17,8%	30,1%
Mã số, max_features = 1,0	12	16,9%	38,3%	13,3%	10,8%	17,7%	29,0%



Hình 4.2. *MAPE tổng và MAPE xe trên 2 tỷ theo cách mã hóa*

Kết luận: giữ one-hot. One-hot cho MAPE thấp nhất, và chênh rõ nhất ở hai đầu thang giá. Lý do: với one-hot, **một** phép chia tách thẳng một dòng xe (ví dụ Mercedes G Class) khỏi phần còn lại; với mã số thứ tự, các nhãn xếp theo bảng chữ cái nên dòng xe sang và xe rẻ nằm xen kẽ, cây phải chia nhiều lần mới cô lập được một dòng xe, và dòng xe hiếm thì không đủ mẫu để chia tới đó. One-hot cũng hợp với bản tự cài đặt: mỗi cột 0/1 nằm gọn trong 2 bin, còn mã số của hơn 200 dòng xe sẽ bị dồn vào 32 bin.

4.3. Cơ sở lý thuyết của Random Forest

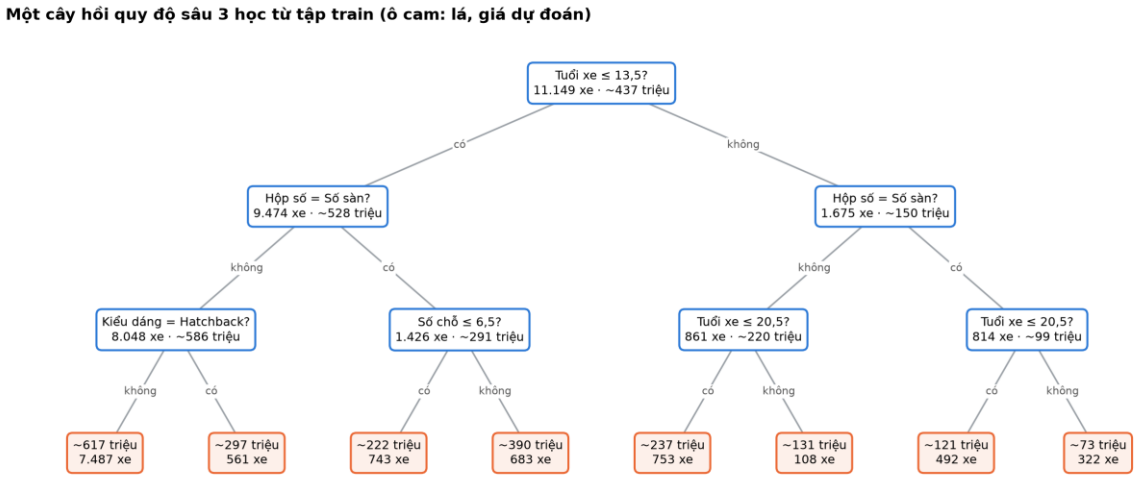
4.3.1. Thành phần: cây hồi quy CART

Cây quyết định chia dữ liệu bằng các câu hỏi có/không, ví dụ "tuổi xe ≤ 13,5?", "hộp số là số sàn?". Tại mỗi nút, thuật toán thử các đặc trưng và ngưỡng, chọn phép chia làm **giảm tổng bình phương sai số (SSE)** nhiều nhất. Với nút có n mẫu, tổng nhãn S, chia thành nhánh trái (n_L mẫu, tổng S_L) và nhánh phải (n_R, S_R):

$$\text{gain} = \frac{S_L^2}{n_L} + \frac{S_R^2}{n_R} - \frac{S^2}{n}$$

Công thức này là mức giảm SSE khi thay giá trị dự đoán chung của nút bằng trung bình riêng của từng nhánh. Lặp lại cho đến khi đạt độ sâu tối đa hoặc nút quá ít mẫu; giá trị dự đoán ở mỗi lá là **trung bình log giá** của các xe trong lá.

Hình 4.3 là một cây **thật** độ sâu 3 học từ tập train. Câu hỏi đầu tiên cây chọn là tuổi xe (≤ 13,5 năm), tiếp theo là hộp số số sàn, rồi kiểu dáng và số chỗ. Chỉ với 3 tầng câu hỏi, cây đã tách được nhóm xe khoảng 617 triệu (xe đời mới, số tự động, không phải hatchback) khỏi nhóm khoảng 73 triệu (xe cũ hơn 20 năm, số sàn). Cây thật trong rừng sâu tới 30 tầng và có hàng nghìn nút.



Hình 4.3. *Một cây hồi quy độ sâu 3 học từ tập train (ô cam: lá, kèm giá dự đoán)*

Vấn đề của một cây: cây sâu học thuộc dữ liệu train (độ chệch thấp) nhưng **phương sai cao**: đổi vài mẫu là cấu trúc cây thay đổi hẳn, và dự đoán trên dữ liệu mới kém ổn định.

4.3.2. Rừng ngẫu nhiên

Random Forest [3] giảm phương sai bằng cách lấy trung bình nhiều cây **khác nhau**:

1. **Bootstrap:** mỗi cây học trên một mẫu lấy ngẫu nhiên **có hoàn lại** từ train, cùng kích thước. Do lấy có hoàn lại, mỗi mẫu bootstrap chứa khoảng 63% số xe khác nhau (xác suất một xe không được chọn sau n lần bốc là $(1 - 1/n)^n \approx e^{-1} \approx 37\%$).
2. **Chọn ngẫu nhiên đặc trưng:** mỗi lần chia nút chỉ xét một tỉ lệ `max_features` đặc trưng ngẫu nhiên, để các cây ít giống nhau.
3. **Lấy trung bình** dự đoán của tất cả T cây:

$$\hat{y}(x) = \frac{1}{T}\sum_{t=1}^{T} f_t(x)$$

Trung bình của nhiều cây **ít tương quan** có phương sai nhỏ hơn hẳn một cây, trong khi độ chệch gần như không đổi. Đó là lý do bước 2 quan trọng: nếu mọi cây đều chọn tuổi xe ở gốc, các cây sẽ giống nhau và lấy trung bình không giúp được nhiều.

Out-of-bag (OOB): khoảng 37% số xe không nằm trong mẫu bootstrap của một cây. Dự đoán mỗi xe bằng những cây "chưa thấy" nó cho ra một **ước lượng sai số khách quan mà không cần tập validation riêng**.

Bảng 4.5. *Siêu tham số của Random Forest*

Siêu tham số	Tác dụng
`n_estimators`	Số cây; nhiều hơn thì ổn định hơn, đến một mức thì bão hòa
`max_depth`	Độ sâu tối đa của cây
`min_samples_leaf`	Số mẫu tối thiểu ở mỗi lá; lớn hơn thì cây "mượt" hơn, ít học thuộc hơn
`max_features`	Tỉ lệ đặc trưng xét ở mỗi lần chia; nhỏ hơn thì các cây đa dạng hơn

4.4. Random Forest: bản thư viện

Dùng `sklearn.ensemble.RandomForestRegressor`, tinh chỉnh bằng **RandomizedSearchCV**: thử ngẫu nhiên 12 cấu hình trong không gian `max_depth` ∈ {20, 30, không giới hạn}, `min_samples_leaf` ∈ {1, 2, 3, 5}, `max_features` ∈ {0,33; 0,5; 0,8}, với 300 cây, đánh giá bằng **3-fold CV phân tầng theo phân khúc giá** trên train (RMSE trên log giá, tương đương sai số tương đối nên nhất quán với MAPE). Tìm kiếm mất khoảng 3,5 phút.

Bảng 4.6. *Năm cấu hình Random Forest tốt nhất (thư viện, 3-fold CV)*

n_estimators	min_samples_leaf	max_features	max_depth	CV RMSE (log)
300	**1**	**0,50**	**30**	**0,2692**
300	1	0,33	30	0,2701
300	2	0,80	30	0,2763
300	2	0,50	30	0,2766
300	2	0,33	30	0,2793

Cấu hình tốt nhất: **300 cây, `max_depth = 30`, `min_samples_leaf = 1`, `max_features = 0,5`**; huấn luyện lại trên toàn bộ train mất khoảng 9 giây, **OOB R^2 (log) = 0,896**. Các cấu hình tốt đều có cây sâu và lá nhỏ: dữ liệu có nhiều tổ hợp hãng-dòng-đời xe riêng biệt nên mô hình cần đủ độ phức tạp để học từng nhóm; việc lấy trung bình 300 cây đã kiểm soát phương sai.

4.5. Random Forest: bản tự cài đặt bằng NumPy

4.5.1. Cấu trúc

Bảng 4.7. *Các lớp của Random Forest tự cài đặt*

Lớp	Nhiệm vụ
`Binner`	Rời rạc hóa mỗi đặc trưng thành tối đa 32 bin theo phân vị (fit trên train)
`RegressionTreeScratch`	Cây hồi quy CART: tìm phép chia tốt nhất theo công thức gain, dựng cây bằng ngăn xếp (không đệ quy)
`RandomForestScratch`	Bootstrap + chọn ngẫu nhiên đặc trưng + lấy trung bình; tính OOB và độ quan trọng đặc trưng

Tăng tốc bằng histogram. Cách cơ bản phải sắp xếp từng cột ở mỗi nút để thử mọi ngưỡng, rất chậm khi viết bằng Python. Thay vào đó, mỗi đặc trưng được chia sẵn thành tối đa 32 bin **một lần** (kỹ thuật của LightGBM [5] và HistGradientBoosting). Tại mỗi nút chỉ cần đếm số mẫu và tổng nhãn trong từng bin (`np.bincount`), rồi cộng dồn (`np.cumsum`) là có n_L, S_L của **mọi ngưỡng cùng lúc**, thay vào công thức gain. Cột one-hot chỉ có 2 giá trị nên không mất thông tin; cột số mất rất ít độ chi tiết.

Độ quan trọng đặc trưng = tổng gain mà đặc trưng mang lại ở mọi nút của mọi cây, chuẩn hóa về tổng bằng 1 (cùng định nghĩa với `sklearn`).

4.5.2. Kiểm chứng thành phần: một cây

Trước khi dựng rừng, so sánh **một cây** tự viết (xét mọi đặc trưng ở mỗi nút) với `DecisionTreeRegressor` khi dùng **cùng tham số** (lá ≥ 5 mẫu, không giới hạn độ sâu):

Bảng 4.8. *Kiểm chứng cây hồi quy tự cài đặt*

Mô hình	Số nút	R^2 (log) train	R^2 (log) test	MAPE test
Một cây – thư viện	3.481	0,929	0,842	19,6%
Một cây – tự cài đặt	3.429	0,927	0,851	18,6%

Hai cây có số nút và R^2 gần như nhau, nên **lớp cây tự viết đúng**. Khoảng cách giữa R^2 train (0,93) và R^2 test (0,84-0,85) chính là **phương sai cao** của một cây. Bảng này dùng tập test chỉ để kiểm tra tính đúng của code, không để chọn tham số.

4.5.3. Tinh chỉnh bằng OOB

Bản tự viết chậm hơn thư viện nhiều, nên thay vì CV (huấn luyện 3-5 lần mỗi cấu hình), dùng **OOB RMSE**: mỗi cấu hình chỉ huấn luyện **một lần** với 30 cây. Thử 6 cấu hình trong cùng không gian tham số, mất khoảng 3 phút. OOB dùng được ở đây vì

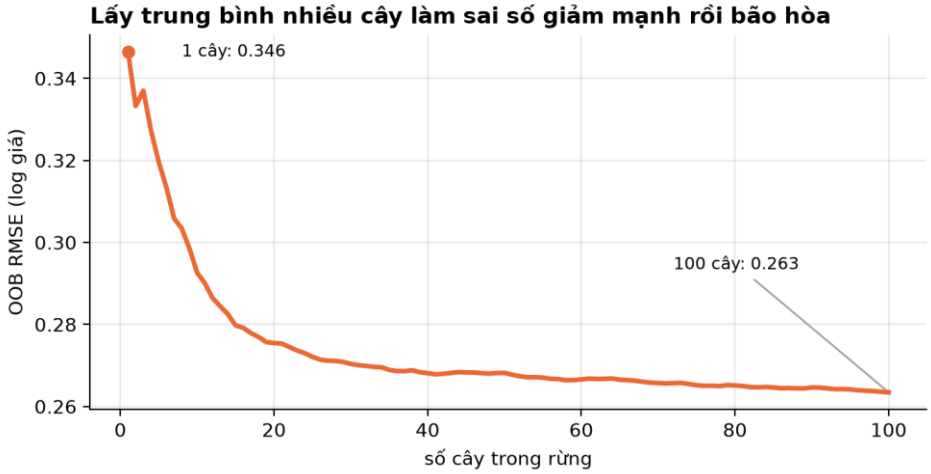
mỗi xe chỉ xuất hiện một lần trong train (mục 4.7 giải thích vì sao OOB **không** dùng được khi oversampling).

Bảng 4.9. *Tinh chỉnh Random Forest tự cài đặt bằng OOB (30 cây mỗi cấu hình)*

max_depth	min_samples_leaf	max_features	OOB RMSE (log)	OOB R^2
30	**2**	**0,50**	**0,2704**	**0,886**
30	2	0,33	0,2741	0,883
30	3	0,33	0,2779	0,879
20	1	0,33	0,2938	0,865
20	2	0,33	0,2946	0,864
20	3	0,50	0,2972	0,862

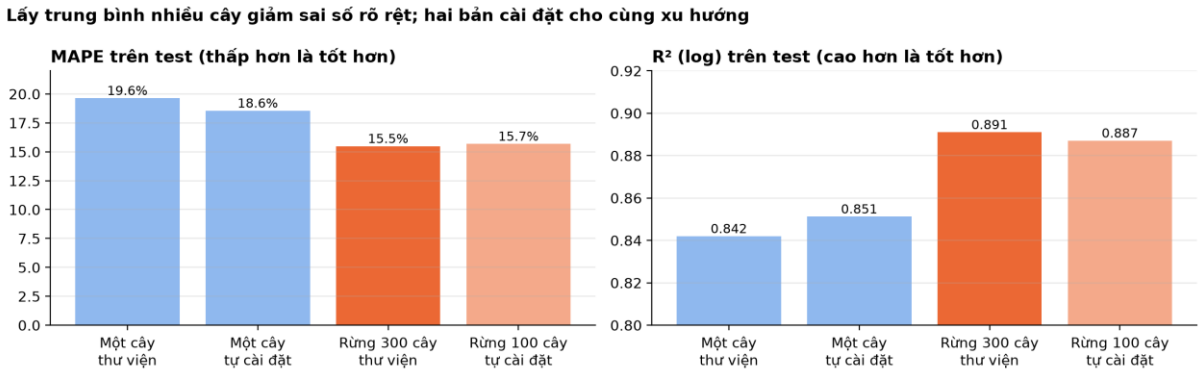
4.5.4. Từ một cây đến một rừng

Với cấu hình tốt nhất, huấn luyện **100 cây** mất khoảng 2,4 phút, **OOB R^2 (log) = 0,892** (bản thư viện 0,896). Hình 4.4 theo dõi OOB RMSE khi số cây tăng dần: sai số giảm rất nhanh trong vài chục cây đầu (từ 0,346 với 1 cây) rồi **bão hòa** quanh 0,263; thêm cây sau điểm đó chỉ tốn thời gian.



Hình 4.4. *OOB RMSE của rừng tự cài đặt theo số cây*

Hình 4.5 so sánh một cây và cả rừng trên tập test: lấy trung bình nhiều cây giảm MAPE từ 18,6-19,6% xuống 15,5-15,7% và tăng R^2 từ khoảng 0,85 lên 0,89. Bản tự cài đặt đi theo đúng xu hướng của bản thư viện ở cả hai mức.



Hình 4.5. *Một cây so với cả rừng trên tập test (hai bản cài đặt)*

Bảng 4.10. *Một cây so với cả rừng trên tập test*

Mô hình	MAPE	MedAPE	R^2 (log)	Lệch ≤ 20%
Một cây – thư viện	19,6%	9,9%	0,842	75%
Một cây – tự cài đặt	18,6%	9,5%	0,851	75%
Rừng 300 cây – thư viện	**15,5%**	**9,0%**	**0,891**	**80%**
Rừng 100 cây – tự cài đặt	15,7%	9,0%	0,887	80%

4.6. Mô hình so sánh: Ridge Regression

Ridge [1] dự đoán log giá bằng **tổ hợp tuyến tính** của các đặc trưng và tìm trọng số bằng cách tối thiểu hóa sai số bình phương cộng một **khoản phạt L2**:

$$J(w) = \frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \frac{\alpha}{2}\sum_{j=1}^{p} w_j^2 \,, \hat{y} = w_0 + \sum_{j=1}^{p} w_j\, x_j$$

Khoản phạt giữ các hệ số ổn định khi có nhiều cột one-hot tương quan (hãng Toyota và dòng Toyota Vios luôn đi cùng nhau). Dùng `sklearn.linear_model.Ridge`, chọn α trên lưới 25 giá trị từ 10^{-3} đến 10^{3} bằng **GridSearchCV + 5-fold CV** trên train: **α = 0,56**, CV RMSE (log) = 0,288.

Vai trò của Ridge trong đề tài: là **mô hình tuyến tính tốt nhất có thể** trên cùng dữ liệu (đã có `age²`, `log(km)` để biểu diễn khấu hao cong, đã được tinh chỉnh như RF). Phần chênh lệch giữa Ridge và Random Forest chính là đóng góp của **quan hệ phi tuyến và tương tác** giữa các đặc trưng. Vì Ridge cùng tập train/test, cùng thước đo và cũng được tinh chỉnh bằng CV, so sánh là **công bằng**.

4.7. Kỹ thuật xử lý mất cân bằng

Ba kỹ thuật được thử trên chính mô hình cuối (Random Forest thư viện, tham số tốt nhất), so với huấn luyện bình thường:

Bảng 4.11. *Các kỹ thuật xử lý mất cân bằng được thử nghiệm*

Kỹ thuật	Cách làm (chỉ trong phần train của mỗi fold)
Không cân bằng	Huấn luyện bình thường
Trọng số mẫu	Mẫu ở phân khúc hiếm có trọng số lớn hơn: 1/√(tần suất phân khúc), chuẩn hóa về trung bình 1
Random Oversampling	Nhân bản ngẫu nhiên mẫu của phân khúc hiếm đến tối thiểu 25% quy mô phân khúc lớn nhất
Random Undersampling	Bỏ bớt mẫu ở phân khúc đông, mỗi phân khúc giữ tối đa 8 lần số mẫu của phân khúc hiếm nhất

Cách đánh giá: 5-fold CV phân tầng theo phân khúc giá **chỉ trên tập train**. Việc cân bằng chỉ áp dụng cho phần train của từng fold; phần validation **giữ nguyên phân phối thật**. Cân bằng trước khi chia sẽ làm bản sao của cùng một xe nằm ở cả hai phía và cho điểm cao giả tạo.

Vì sao dùng CV chứ không dùng OOB: khi oversampling, bản sao của một xe có thể nằm trong mẫu bootstrap của một cây trong khi bản gốc là mẫu "ngoài túi" của chính cây đó. OOB khi đó đánh giá trên xe mà mô hình đã thấy và cho điểm đẹp giả tạo. CV tách validation **trước** khi nhân bản nên không gặp vấn đề này. Thí nghiệm cũng so sánh học trên giá gốc với học trên log giá. Tập test không được dùng ở bước này.

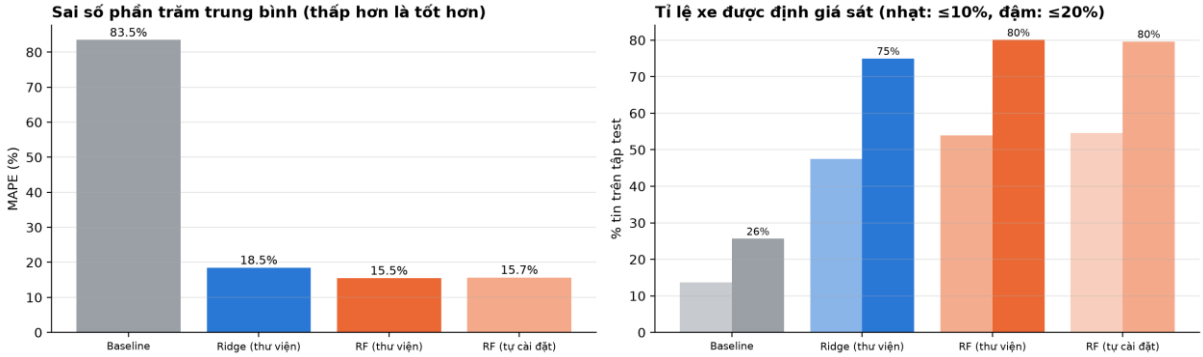
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả tổng thể trên tập test

Đây là lần **duy nhất** tập test được dùng để đánh giá. Mọi mô hình học trên cùng tập train (11.149 tin) và được đánh giá trên cùng 2.788 tin test.

Bảng 5.1. *Kết quả trên tập test*

Mô hình	MAPE	MedAPE	MAE (tr)	RMSE (tr)	R^2 (log)	±10%	±20%	Huấn luyện
Baseline (trung vị)	83,5%	40,0%	337	724	0,00	14%	26%	–
Ridge – thư viện (so sánh)	18,5%	10,7%	120	411	0,873	48%	75%	0,04 s
Random Forest – thư viện	**15,5%**	**9,0%**	**102**	**372**	**0,891**	**54%**	**80%**	**9 s**
Random Forest – tự cài đặt	15,7%	9,0%	106	388	0,887	55%	80%	143 s



Hình 5.1. *So sánh MAPE và tỉ lệ xe được định giá sát giữa các mô hình*

Cách đọc bảng:

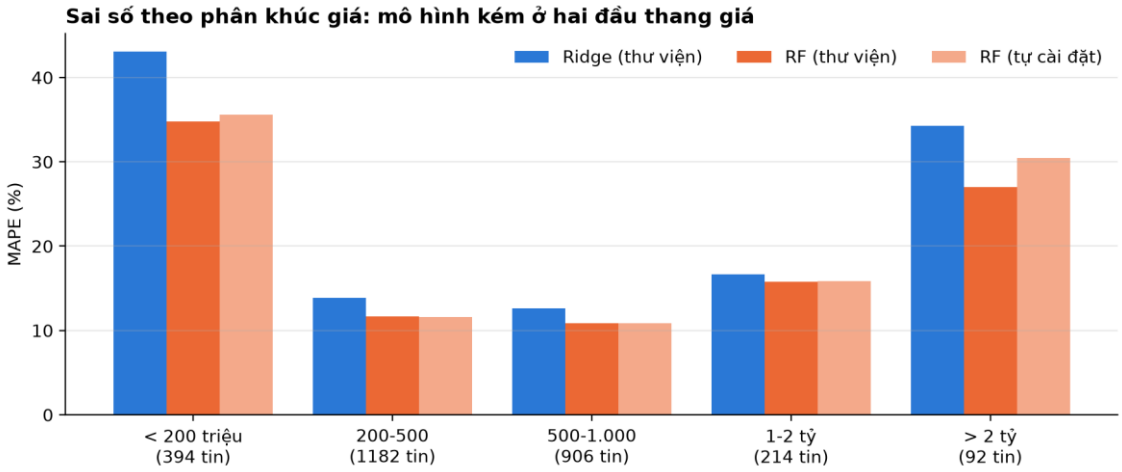
- **Random Forest vượt xa mốc so sánh:** baseline sai trung bình 83,5% và R^2 gần 0; Random Forest giảm còn **15,5%**, R^2 (log) 0,891. Theo thang tham khảo ở Bảng 4.2, đây là mức **tốt** (10-20%).
- **MedAPE 9,0% thấp hơn hẳn MAPE 15,5%:** xe "điển hình" chỉ bị đoán lệch khoảng 9%; một nhóm nhỏ xe bị sai rất nặng (chủ yếu ở hai đầu thang giá, mục 5.2 và 5.4) kéo trung bình lên.
- **RMSE (372 triệu) gấp khoảng 3,6 lần MAE (102 triệu):** RMSE bình phương sai số nên phóng đại lỗi lớn; điều này cho thấy có một số xe bị đoán sai hàng tỷ đồng, chủ yếu là xe sang.
- **Lệch ≤ 20% đạt 80%:** 4 trên 5 xe được định giá sát trong khoảng ±20%; hơn một nửa sát trong ±10%. Ngay cả baseline đoán mù cũng trúng 26% vì nhiều xe có giá quanh mức trung vị, nên chỉ số này phải đọc cạnh baseline.

- **Bản tự cài đặt** gần như trùng bản thư viện về độ chính xác; khác biệt lớn nhất là thời gian huấn luyện (143 giây so với 9 giây).

5.2. Sai số theo phân khúc giá

Bảng 5.2. *MAPE theo phân khúc giá trên tập test*

Phân khúc	% dữ liệu	Baseline	Ridge	RF (thư viện)	RF (tự cài đặt)
< 200 triệu	14%	349,8%	43,0%	34,7%	35,6%
200-500 triệu	42%	38,5%	13,9%	11,7%	11,6%
500-1.000 triệu	33%	30,7%	12,6%	10,9%	10,8%
1-2 tỷ	8%	64,1%	16,7%	15,7%	15,8%
> 2 tỷ	3%	85,4%	34,3%	27,0%	30,4%

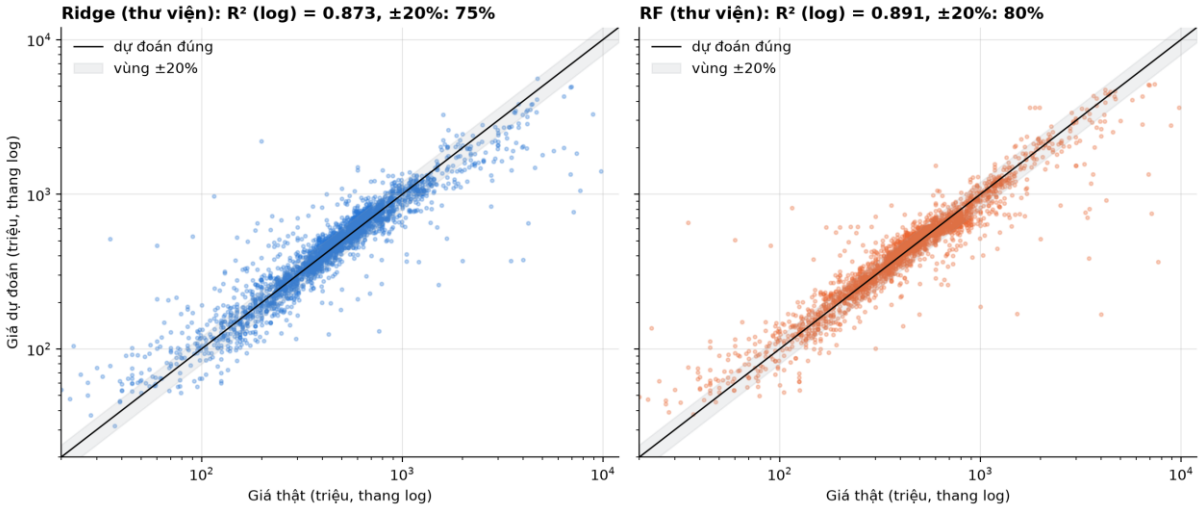


Hình 5.2. *MAPE theo phân khúc giá*

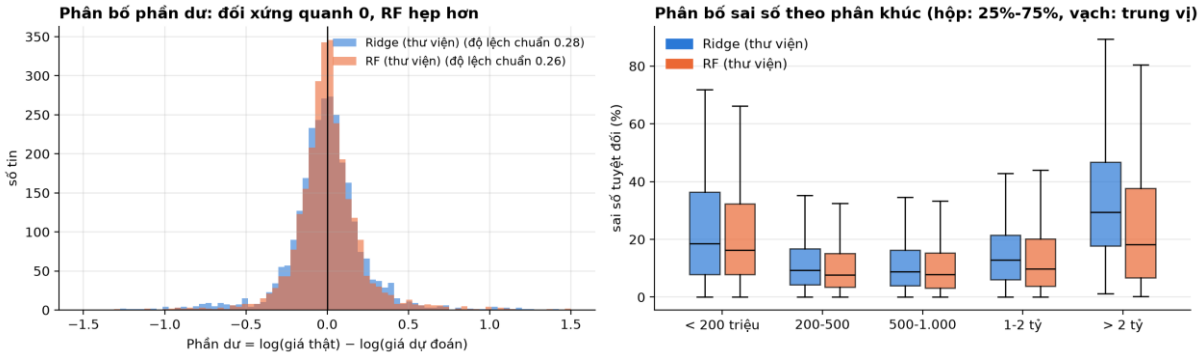
- **Phân khúc giữa (200 triệu - 1 tỷ, khoảng 75% thị trường):** sai số khoảng 11%, đủ tốt để tham khảo giá.
- **Xe dưới 200 triệu (35%):** giá thấp nên chênh vài chục triệu (tình trạng thực tế, thương lượng) đã là phần trăm lớn; xe đời cũ rất khác nhau về tình trạng mà dữ liệu không có; và còn sót tin ghi giá trả trước (mục 5.4).
- **Xe trên 2 tỷ (27%):** chỉ 3% dữ liệu và thiếu thông tin quyết định giá xe sang (phiên bản, trang bị).
- Random Forest tốt hơn Ridge ở **mọi phân khúc**. Baseline có sai số thấp nhất ở 500-1.000 triệu chỉ vì giá trung vị nằm sẵn trong khoảng này, cho thấy vì sao hai đầu thang giá luôn khó hơn.

5.3. Dự đoán so với giá thật và phần dư

Trên Hình 5.3, mỗi điểm là một xe trong tập test; trục dùng thang log nên khoảng cách tới đường chéo chính là sai số phần trăm. Phần dư của cả hai mô hình đối xứng quanh 0 (không lệch hệ thống), Random Forest có phân bố hẹp hơn. Biểu đồ hộp cho thấy sai số tăng ở hai đầu thang giá.



Hình 5.3. *Giá dự đoán so với giá thật trên tập test (thang log)*



Hình 5.4. *Phân bố phần dư và sai số tuyệt đối theo phân khúc*

5.4. Những xe bị đoán sai nhiều nhất

Để hiểu sai số lớn đến từ đâu, xem cụ thể 10 xe Random Forest (thư viện) đoán lệch nhiều nhất trên tập test:

Bảng 5.3. *Mười xe bị đoán sai nhiều nhất trên tập test (giá: triệu đồng)*

Hãng	Dòng xe	Năm	Tình trạng	Số km	Giá thật	Dự đoán	Lệch
Hyundai	Starex	2023	Đã dùng	20.000	35	655	+1.771%
Isuzu	Hi lander	2023	Đã dùng	100.000	60	505	+742%
Toyota	4 Runner	2025	Đã dùng	9.000	115	819	+612%
Daewoo	Matiz	2016	Đã dùng	300.000	23	136	+493%
Hyundai	Accent	2026	Mới	0	90	464	+416%

Hãng	Dòng xe	Năm	Tình trạng	Số km	Giá thật	Dự đoán	Lệch
Mitsubishi	Xforce	2025	Đã dùng	18.000	170	612	+260%
MINI	Cooper	2025	Đã dùng	91.000	199	612	+208%
Hyundai	Tucson	2021	Đã dùng	44.000	190	567	+198%
Ford	Ranger	2024	Đã dùng	36.750	225	616	+174%
Toyota	Wigo	2023	Đã dùng	12.000	148	379	+157%

Nhận xét: sai số lớn nhất phần nhiều đến từ **nhãn sai còn sót**, không phải từ mô hình. Phần lớn là xe **đời 2021-2026** nhưng rao chỉ 35-225 triệu (Hyundai Starex 2023 giá 35 triệu, Hyundai Accent 2026 **xe mới** giá 90 triệu): gần như chắc chắn là **giá trả trước / đặt cọc**. Luật lọc ở mục 3.5 chỉ áp dụng cho nhóm có ít nhất 3 xe cùng hãng-dòng-năm, nên các dòng xe ít tin lọt qua. Với những xe này, giá mô hình dự đoán (400-650 triệu) hợp lý hơn giá rao. Isuzu Hi lander đời "2023" nhiều khả năng là nhập sai năm sản xuất (dòng xe này đã ngừng bán từ lâu). Hệ quả: MAPE, nhất là phân khúc dưới 200 triệu, bị **đội lên** bởi các tin này.

5.5. Kiểm định ý nghĩa thống kê bằng bootstrap

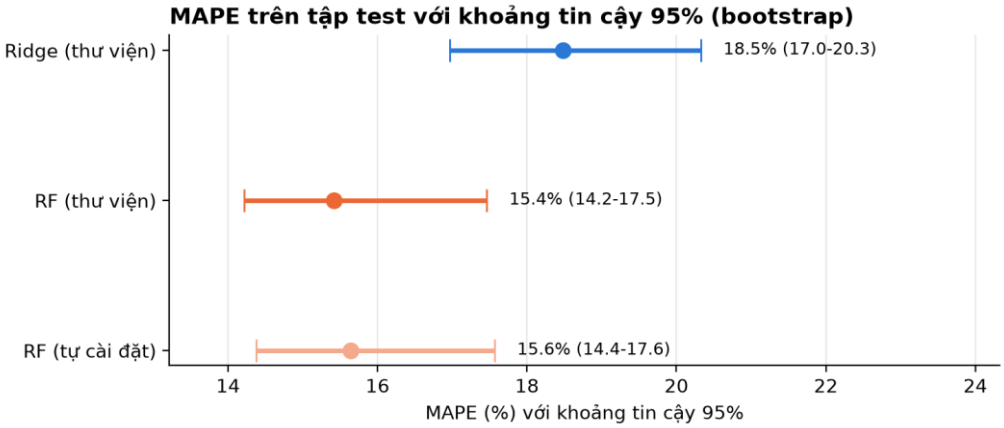
Khoảng tin cậy là gì? MAPE 15,5% được đo trên 2.788 xe cụ thể. Nếu tình cờ lấy 2.788 xe khác, con số có thể là 15,1% hay 16,0%. Khoảng tin cậy 95% cho biết con số này có thể dao động trong khoảng nào nếu đánh giá lại trên một tập dữ liệu khác.

Cách ước lượng bằng bootstrap [7]: từ 2.788 xe test, bốc ngẫu nhiên 2.788 xe **có hoàn lại** để được một "tập test giả", tính lại MAPE; lặp lại 1.000 lần, được 1.000 giá trị MAPE; bỏ 2,5% thấp nhất và 2,5% cao nhất, khoảng còn lại là **khoảng tin cậy 95%**.

So sánh hai mô hình bằng bootstrap ghép cặp: không nên chỉ nhìn hai khoảng tin cậy có chồng nhau không (cách này quá khắt khe). Cách đúng là trong mỗi lần bốc, tính **hiệu MAPE của hai mô hình trên cùng các xe đó**. Nếu khoảng tin cậy 95% của hiệu **không chứa 0**, chênh lệch có ý nghĩa thống kê; nếu **chứa 0**, chênh lệch có thể chỉ là ngẫu nhiên.

Bảng 5.4. *Hiệu MAPE giữa các mô hình (điểm %), bootstrap ghép cặp 1.000 lần*

So sánh	2,5%	Trung vị	97,5%	Có ý nghĩa?
Ridge − Random Forest (thư viện)	2,08	2,97	3,97	**Có** (cả khoảng > 0)
RF tự cài đặt − RF thư viện	−0,04	0,19	0,46	Không (khoảng chứa 0)

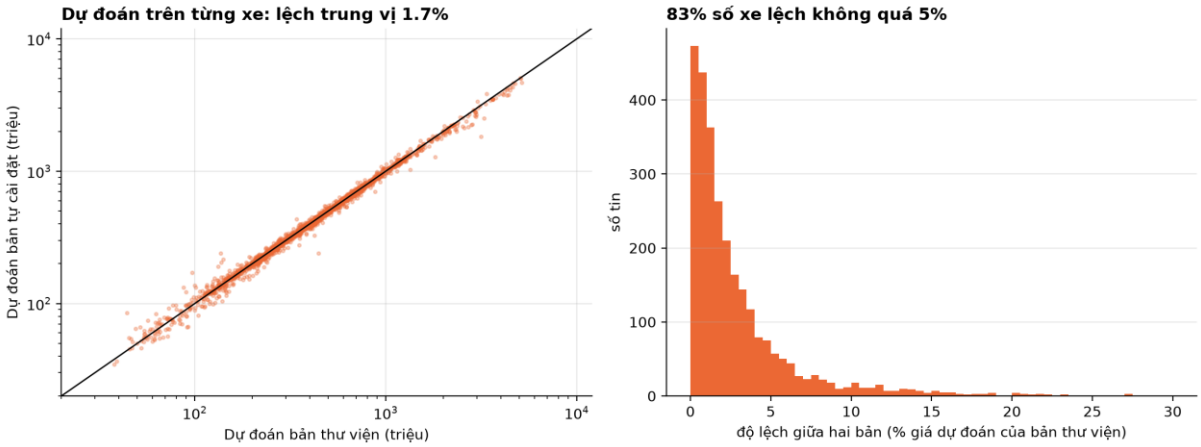


Hình 5.5. *MAPE trên tập test với khoảng tin cậy 95% (bootstrap)*

Kết luận: Random Forest tốt hơn Ridge khoảng **3 điểm %** và chênh lệch **có ý nghĩa thống kê** (KTC 95%: 2,1-4,0). Rừng tự cài đặt và rừng thư viện **không khác nhau có ý nghĩa**.

5.6. Rừng tự cài đặt so với rừng thư viện trên từng xe

Nếu hai bản cài đặt cùng một thuật toán, dự đoán của chúng phải gần như trùng nhau trên **từng xe**, không chỉ ở con số trung bình. Hình 5.6: các điểm bám sát đường chéo, độ lệch trung vị giữa hai bản là **1,7%**, và **83% số xe** lệch không quá 5%.



Hình 5.6. *Dự đoán của rừng tự cài đặt so với rừng thư viện trên từng xe trong tập test*

Bảng 5.5. *Tóm tắt so sánh hai bản Random Forest*

Tiêu chí	Thư viện	Tự cài đặt
MAPE test	15,5%	15,7%
OOB R^2 (log)	0,896	0,892
Số cây / cách chọn tham số	300 / CV	100 / OOB
Tìm ngưỡng chia	chính xác, mọi giá trị	histogram 32 bin
Thứ hạng độ quan trọng đặc trưng	tuổi, hộp số, hãng, dòng xe...	giống hệt

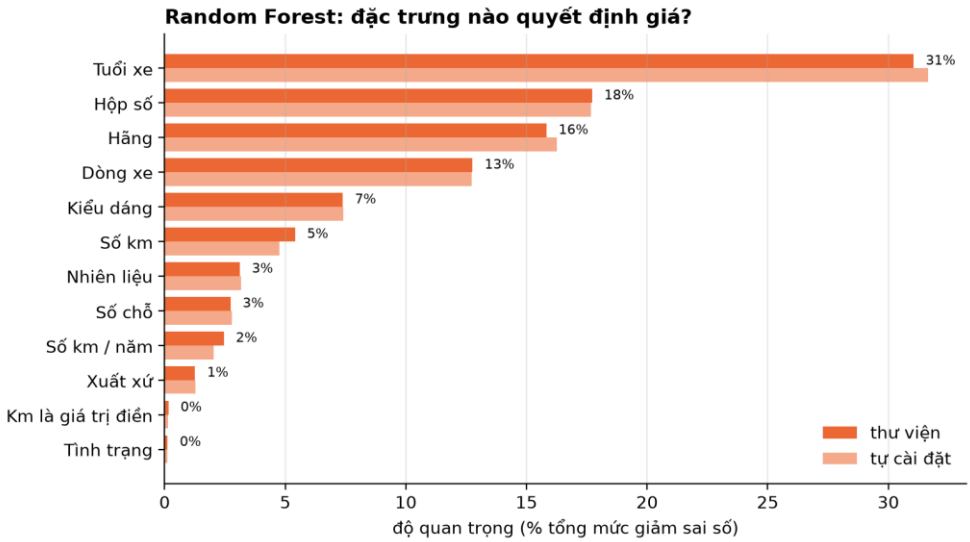
Tiêu chí	Thư viện	Tự cài đặt
Thời gian huấn luyện	9 giây (C/Cython, nhiều lõi)	143 giây (Python, một lõi)

Chênh lệch nhỏ giữa hai bản đến từ việc chia đặc trưng thành 32 bin, dùng 100 cây thay vì 300, và cấu hình tinh chỉnh bằng OOB hơi khác (`min_samples_leaf = 2`). Rõ nhất ở xe trên 2 tỷ (30,4% so với 27,0%), nơi ít mẫu nên bin thô và lá lớn hơn ảnh hưởng nhiều hơn.

5.7. Diễn giải mô hình

5.7.1. Độ quan trọng theo mức giảm sai số (impurity)

Một đặc trưng phân loại bị one-hot thành nhiều cột, nên độ quan trọng của mọi cột thuộc cùng một đặc trưng gốc được cộng lại:

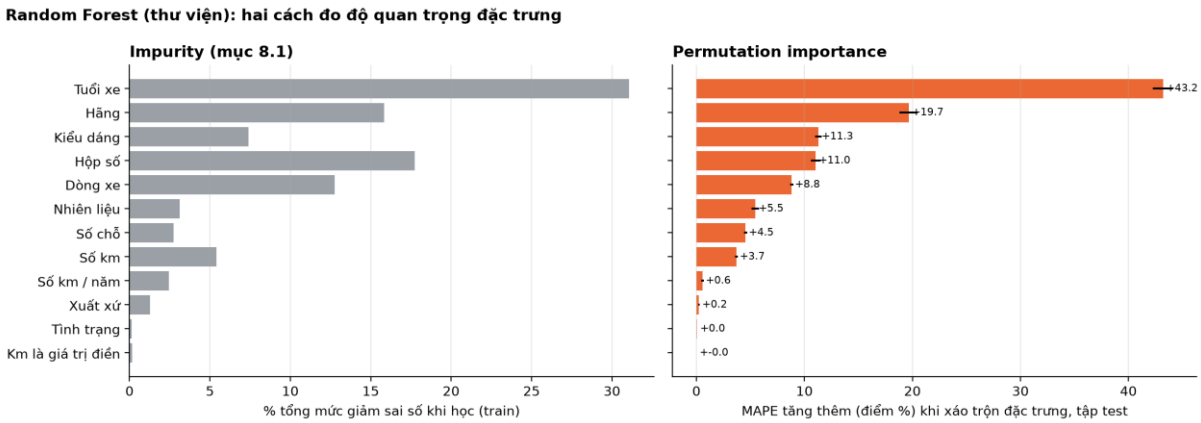


Hình 5.7. *Độ quan trọng đặc trưng (impurity): bản thư viện và bản tự cài đặt*

Tuổi xe đứng đầu (31%), sau đó là hộp số (18%), hãng (16%), dòng xe (13%). Hai bản cài đặt cho **cùng thứ hạng** và gần như cùng tỉ lệ, thêm một bằng chứng rừng tự viết hoạt động đúng. Tuy nhiên cách đo này có hai điểm yếu đã biết [10]: tính trên train (kể cả những phép chia chỉ học thuộc nhiễu), và **bị lệch khi các đặc trưng tương quan**. Ví dụ xe số sàn thường là xe đời cũ, nên hộp số và tuổi xe mang trùng thông tin; cây chọn đặc trưng nào để chia phần nhiều là ngẫu nhiên.

5.7.2. Kiểm tra lại bằng permutation importance

Permutation importance [3] hỏi trực tiếp: nếu **xáo trộn ngẫu nhiên** một đặc trưng trên tập test (phá vỡ quan hệ của nó với giá, giữ nguyên phân phối), MAPE tăng bao nhiêu? Đo trên **đặc trưng gốc** (xáo trộn cả cột dòng xe rồi mới mã hóa), lặp 5 lần mỗi đặc trưng. Tập test chỉ dùng để diễn giải mô hình đã chọn, không để chọn gì.



Hình 5.8. *Hai cách đo độ quan trọng đặc trưng của Random Forest*

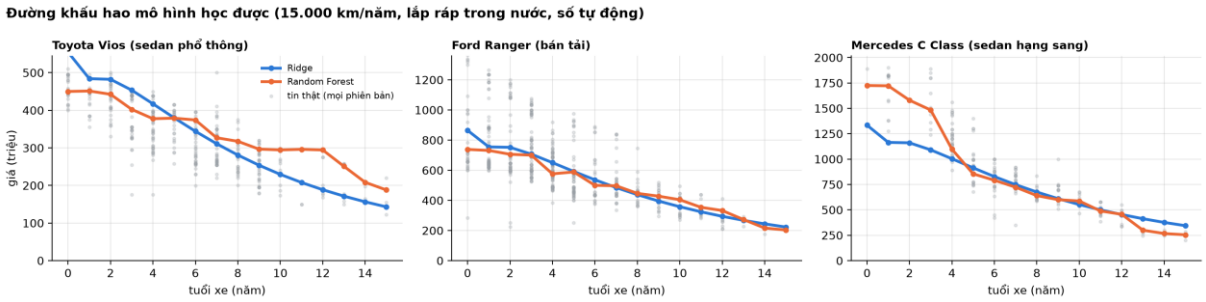
Bảng 5.6. *Độ quan trọng đặc trưng của Random Forest (thư viện) theo hai cách đo*

Đặc trưng	Impurity (%)	Permutation: MAPE tăng (điểm %)	Hạng (impurity → permutation)
Tuổi xe	31,0	43,2	1 → 1
Hãng	15,8	19,7	3 → 2
Kiểu dáng	7,4	11,3	5 → 3
Hộp số	17,7	11,0	**2 → 4**
Dòng xe	12,8	8,8	4 → 5
Nhiên liệu	3,1	5,5	7 → 6
Số chỗ	2,7	4,6	8 → 7
Số km	5,4	3,7	6 → 8
Số km / năm	2,5	0,6	9 → 9
Xuất xứ, tình trạng, km là giá trị điền	≤ 1,3	≤ 0,2	cuối

- **Tuổi xe** quan trọng nhất theo cả hai cách: khấu hao là yếu tố lớn nhất quyết định giá xe cũ.
- **Hộp số tụt từ hạng 2 xuống hạng 4:** một phần độ quan trọng impurity của nó là thông tin **trùng với tuổi xe**. Khi xáo trộn hộp số, tuổi xe vẫn giữ phần thông tin chung đó.
- **Hãng lên hạng 2:** mức giá chung của thương hiệu là thông tin mà các đặc trưng khác không thay thế được. Dòng xe xếp sau hãng vì một phần thông tin đã có trong hãng, kiểu dáng, số chỗ.
- **Số km/năm, xuất xứ, tình trạng, cờ km là giá trị điền** gần như bằng 0: tình trạng mới/cũ đã nằm trong tuổi = 0 và km = 0.
- Xáo trộn từng đặc trưng độc lập có thể tạo tổ hợp không thực tế (xe 0 tuổi đi 200.000 km), nên con số dùng để **xếp hạng**, không phải tác động nhân quả.

5.7.3. Đường khấu hao

Lấy ba mẫu xe phổ biến ở ba phân khúc, cố định mọi thuộc tính, chỉ thay đổi tuổi xe (giả định 15.000 km/năm; tuổi 0 là xe mới, 0 km), rồi xem hai mô hình dự đoán giá:



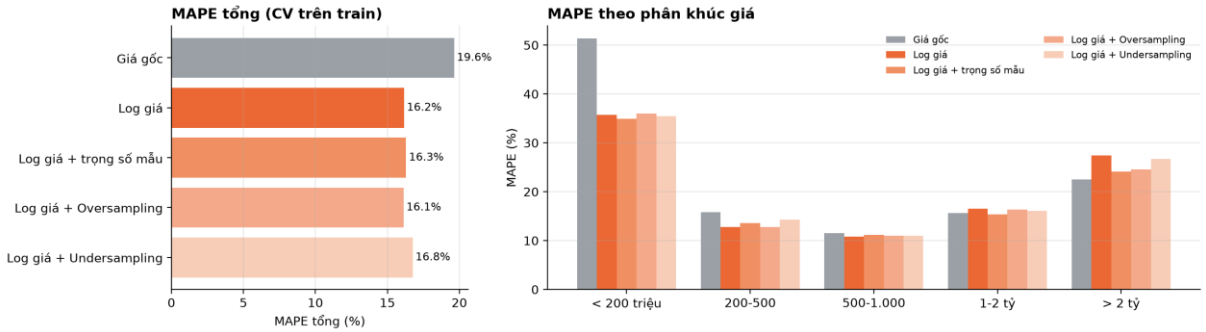
Hình 5.9. *Giá dự đoán theo tuổi xe của ba mẫu xe ở ba phân khúc*

- **Ridge** cho đường khấu hao trơn và **cùng một hình dạng** cho mọi mẫu xe (chỉ dịch lên xuống), vì mô hình tuyến tính trên log giá giả định mọi xe mất giá cùng một tỉ lệ % mỗi năm.
- **Random Forest** cho đường bậc thang với **hình dạng khác nhau giữa các dòng xe**: xe sang (Mercedes C Class) mất giá rất nhanh trong 3-4 năm đầu, khác hẳn xe phổ thông. Đây chính là **tương tác** giữa dòng xe và tuổi xe mà Ridge không biểu diễn được, và là lý do Random Forest chính xác hơn 3 điểm %.

5.8. Thí nghiệm log giá và xử lý mất cân bằng

Bảng 5.7. *Thí nghiệm trên Random Forest (5-fold CV trên tập train)*

Phương án	Cỡ train/fold	MAPE	MedAPE	< 200 tr	200-500	500-1.000	1-2 tỷ	> 2 tỷ
Giá gốc	8.919	19,6%	9,7%	51,3%	15,8%	11,5%	15,6%	22,5%
Log giá (chọn)	**8.919**	**16,2%**	**9,1%**	**35,7%**	**12,8%**	**10,8%**	**16,5%**	**27,4%**
Log + trọng số mẫu	8.919	16,3%	8,8%	34,9%	13,6%	11,2%	15,3%	24,1%
Log + Oversampling	9.830	16,1%	8,7%	36,0%	12,8%	11,0%	16,3%	24,5%
Log + Undersampling	6.925	16,8%	9,5%	35,5%	14,3%	11,0%	16,0%	26,7%



Hình 5.10. *MAPE tổng và MAPE theo phân khúc của các phương án (CV trên train)*

Log giá tốt hơn rõ rệt. MAPE tổng giảm từ 19,6% xuống 16,2%; phân khúc dưới 200 triệu cải thiện mạnh nhất (51% → 36%). Học trên giá gốc thì sai số bình phương bị xe đắt chi phối, nên mô hình bỏ qua sai số tương đối ở xe rẻ. Đổi lại, giá gốc tốt hơn ở xe trên 2 tỷ (22,5% so với 27,4%), cũng vì lý do đó.

- **MAPE tổng** của ba kỹ thuật cân bằng nằm trong khoảng 16,1-16,8%, so với 16,2% khi không cân bằng.
- **Trọng số mẫu** và **oversampling** giảm sai số ở xe trên 2 tỷ khoảng 3 điểm % (27,4% → 24,1% và 24,5%). Trọng số mẫu đổi lại bằng sai số cao hơn ở phân khúc đông nhất 200-500 triệu (12,8% → 13,6%); oversampling gần như không làm phân khúc nào kém đi.
- **Undersampling** bỏ khoảng 22% dữ liệu train, MAPE tổng tăng lên 16,8% và gần như không giúp xe sang: bỏ dữ liệu thật thì mất thông tin.
- Dù đã cân bằng, sai số ở xe trên 2 tỷ vẫn cao gấp đôi phân khúc giữa: nguyên nhân chủ yếu là **thiếu thông tin** (phiên bản, trang bị), không chỉ do ít mẫu. Nhân bản dữ liệu không tạo thêm thông tin mới.

Quyết định: mô hình cuối **học trên log giá, không dùng kỹ thuật cân bằng**, vì thước đo chính không đổi và mô hình đơn giản hơn. Nếu ứng dụng ưu tiên định giá xe sang, nên dùng **Random Oversampling** (chỉ trên tập train).

5.9. Ảnh hưởng của tin đăng lại

Bước tiền xử lý đã xóa các tin trùng hoàn toàn, nhưng cùng một chiếc xe có thể được **đăng lại với giá chênh nhẹ**. Khi chia ngẫu nhiên, một bản rơi vào train và bản kia vào test; Random Forest với lá nhỏ chỉ cần "nhớ" giá là đoán đúng. Hai cách kiểm tra:

(1) Trên tập test: tin test nào có một tin train **giống hệt mọi đặc trưng, kể cả số km**?

Bảng 5.8. *Sai số trên nhóm tin có và không có bản sao trong tập train*

Nhóm tin test	Số tin	% test	MAPE Ridge	MAPE RF (TV)	MAPE RF (TCĐ)
Có bản sao trong train	391	14,0%	14,0%	10,9%	10,8%
trong đó xe đã dùng	247	8,9%	14,0%	10,5%	10,7%
Không có bản sao	2.397	86,0%	19,2%	16,2%	16,5%
Toàn bộ test	2.788	100%	18,5%	15,5%	15,7%

(2) CV chia theo nhóm trên tập train: chạy lại 5-fold CV cho Random Forest theo hai cách: chia ngẫu nhiên (`StratifiedKFold`) và chia theo nhóm

(`StratifiedGroupKFold`), trong đó mọi tin giống hệt nhau về đặc trưng luôn nằm **cùng một phía**. Train có 10.175 nhóm trên 11.149 tin.

Bảng 5.9. *MAPE (CV trên train) khi chia ngẫu nhiên và chia theo nhóm*

Cách chia	MAPE	< 200 tr	200-500	500-1.000	1-2 tỷ	> 2 tỷ
Ngẫu nhiên (StratifiedKFold)	16,1%	35,7%	12,8%	10,8%	16,5%	27,6%
Theo nhóm (StratifiedGroupKFold)	16,7%	38,0%	13,0%	10,9%	16,9%	28,7%

Chia theo nhóm làm MAPE tăng khoảng **0,5 điểm %**, rõ nhất ở xe dưới 200 triệu. Mức lạc quan do tin đăng lại là **có nhưng nhỏ** và không đổi kết luận nào. **Ước lượng thận trọng** cho một chiếc xe mô hình chưa từng thấy: MAPE khoảng **16-16,5%**.

5.10. Lưu và sử dụng mô hình

Bảng 5.10. *Các file mô hình đã lưu trong thư mục model/*

File	Nội dung
`preprocessor_rf.joblib`, `preprocessor_ridge.joblib`	Bộ mã hóa của từng mô hình (fit trên train)
`random_forest_sklearn.joblib`, `ridge_sklearn.joblib`	Mô hình thư viện
`random_forest_scratch.npz`	Các cây và ngưỡng bin của rừng tự cài đặt (mảng NumPy, không phụ thuộc lớp trong notebook)
`metadata.json`	Danh sách đặc trưng, bảng gộp nhãn, trung vị số km theo năm, tham số tốt nhất, metric
`price_model.py`	Hàm `load_model` / `predict_price`: nhận thông tin thô của xe, tự dựng đặc trưng như lúc huấn luyện

Khi nạp lại bằng `price_model.py` và truyền thông tin thô của các xe test, dự đoán trùng với notebook (lệch lớn nhất khoảng 10^{-11} triệu). Ví dụ:

Bảng 5.11. *Ví dụ dự đoán giá (triệu đồng)*

Xe	RF (thư viện)	RF (tự cài đặt)	Ridge
Toyota Vios 2018, 80.000 km	303	303	281
Ford Ranger 2021, không rõ số km	515	536	596

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

- Xây dựng hoàn chỉnh quy trình học máy từ đầu đến cuối: thu thập **14.835 tin** từ Chợ Tốt, tiền xử lý còn **13.937 tin** sạch, huấn luyện, đánh giá, diễn giải và đóng gói mô hình để sử dụng.
- **Random Forest** đạt MAPE **15,5%**, MedAPE 9,0%, R^2 (log) 0,891, **80% xe** được định giá lệch không quá 20%; tốt hơn Ridge (18,5%) khoảng 3 điểm %, **có ý nghĩa thống kê** (KTC 95%: 2,1-4,0). Phần chênh này là đóng góp của quan hệ phi tuyến và tương tác giữa các đặc trưng.
- **Random Forest tự cài đặt bằng NumPy** (CART tìm ngưỡng bằng histogram, bootstrap, chọn ngẫu nhiên đặc trưng, OOB, độ quan trọng đặc trưng) được kiểm chứng ở cả mức một cây và cả rừng: MAPE 15,7%, khác bản thư viện không có ý nghĩa thống kê, dự đoán trên từng xe lệch trung vị 1,7%.
- Mọi quyết định đều kiểm chứng bằng CV/OOB trên train: one-hot tốt hơn mã số thứ tự; học trên log giá giảm MAPE từ 19,6% xuống 16,2%; trọng số mẫu và oversampling giảm khoảng 3 điểm % sai số xe trên 2 tỷ; undersampling làm kết quả kém đi.
- Yếu tố quyết định giá (permutation importance): tuổi xe, hãng, kiểu dáng, hộp số, dòng xe.

6.2. Hạn chế

- **Giá rao bán, không phải giá giao dịch:** luôn có phần thương lượng mà mô hình không biết.
- **Thiếu đặc trưng quan trọng:** trang tin có các mục **số đời chủ, còn hạn đăng kiểm, phụ kiện đi kèm** và tiêu đề tin thường ghi **phiên bản xe**, nhưng code thu thập chưa lấy. Đây là nguyên nhân chính của sai số cao ở xe trên 2 tỷ (27%).
- **Nhãn sai còn sót:** luật lọc giá trả trước chỉ áp dụng cho nhóm ≥ 3 xe, nên vẫn còn tin ghi giá trả trước (mục 5.4), làm MAPE phân khúc dưới 200 triệu bị đội lên. Mặt khác, luật này dùng giá trên toàn bộ dữ liệu (là bước sửa nhãn), nên khi dùng thật mô hình vẫn sẽ gặp các tin như vậy.
- **Tin đăng lại:** 14% tin test có bản sao trong train; ước lượng thận trọng khoảng 16-16,5% thay vì 15,5%.
- **Mẫu dữ liệu:** chỉ lấy 750 trang tin mới nhất tại một thời điểm (10/2026), nghiêng về tin đăng gần đây và đại lý đăng nhiều; xe trên 10 tỷ đã bị loại nên mô hình không áp dụng cho siêu xe.

- **Rò rỉ nhẹ trong tiền xử lý:** điền thiếu và đếm nhãn hiếm dùng thống kê trên toàn bộ dữ liệu trước khi chia (chỉ dựa trên đặc trưng, không dùng giá). Rủi ro thấp nhưng cần nêu rõ.

6.3. Hướng phát triển

- Crawl thêm **tiêu đề, mô tả** và các mục số đời chủ, đăng kiểm: trích phiên bản xe, lọc giá trả trước theo từ khóa ("trả trước", "góp", "vay", "cọc").
- Chia train/test theo nhóm (`StratifiedGroupKFold`) làm cách đánh giá chính.
- Thử **Gradient Boosting** (học tuần tự các cây sửa lỗi của nhau), có thể dùng lại lớp cây đã tự cài đặt.
- Sau khi đánh giá, huấn luyện lại trên toàn bộ dữ liệu trước khi triển khai, và huấn luyện lại định kỳ khi thị trường thay đổi.
- Xây dựng giao diện web nhỏ cho phép nhập thông tin xe và nhận giá tham khảo, dựa trên `price_model.py`.

TÀI LIỆU THAM KHẢO

[1] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

[2] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.

[5] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems 30*.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[7] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

[8] Chợ Tốt Xe. Chuyên mục mua bán ô tô. https://xe.chotot.com/mua-ban-oto (truy cập 25/10/2026).

[9] Lewis, C. D. (1982). *Industrial and Business Forecasting Methods*. Butterworths.

[10] Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8, 25.

PHỤ LỤC

A. Cấu trúc mã nguồn

Bảng PL.1. *Cấu trúc thư mục dự án*

Đường dẫn	Nội dung
`src/crawl_data.ipynb`	Thu thập dữ liệu từ Chợ Tốt
`src/visualize_raw_data.ipynb`	Biểu đồ khám phá dữ liệu thô
`src/preprocess_data.ipynb`	Tiền xử lý, tạo `clean_data_oto.csv`
`src/model.ipynb`	Random Forest (thư viện + tự cài đặt), Ridge để so sánh, đánh giá, thí nghiệm, xuất mô hình
`src/figures/`	Các biểu đồ dùng trong báo cáo
`model/price_model.py`	Nạp mô hình và dự đoán giá từ thông tin thô
`slides/`	Slide trình bày

B. Cách chạy

Chạy các notebook với thư mục làm việc là `src` theo thứ tự: `crawl_data` (không bắt buộc, đã có dữ liệu), `preprocess_data` (khoảng 2 phút), `model` (khoảng 15 phút, phần lớn là rừng tự cài đặt). Mọi bước ngẫu nhiên cố định `random_state = 42` nên kết quả tái lập được. Dùng mô hình đã lưu:

```
from price_model import load_model, predict_price
import pandas as pd
xe = pd.DataFrame([{"carbrand": "Toyota", "carmodel": "Vios", "carorigin": "Việt
Nam",
    "condition_ad": "Đã sử dụng", "gearbox": "Tự động", "fuel": "Xăng",
    "cartype": "Sedan", "carseats": 5, "mfdate": 2018, "mileage_km": 80000}])
print(predict_price(load_model("random_forest_sklearn"), xe))   # ~303 triệu
```

C. Phân công công việc

Bảng PL.2. *Phân công công việc trong nhóm*

Thành viên	Công việc
<Họ tên SV 1>	<Nội dung công việc>
<Họ tên SV 2>	<Nội dung công việc>